Networks Unmasked

How Data Travels, What It Reveals, and the Cost of Privacy

DEFINITIVE EDITION



Emil Valiyev

Contents

Preface 3
1 A message needs a path 5
2 Inside the host 10
3 Addresses and neighbors 15
4 Routes and rewritten addresses 19
5 Getting an address and finding a name 23
6 Sockets ports and datagrams 27
7 TCP keeps a conversation in order 30
8 The application shapes the traffic 34
9 What the path reveals 38
10 Interception and manipulation 42
11 What a protected channel establishes 46
12 Protecting names without hiding their limits 50
13 Enforcing a path 53
14 Proxies tunnels and VPNs 57
15 Tor builds a circuit 61
16 Tor at the boundaries 65
17 I2P connects destinations 69
18 Names trust and cost in I2P 74
19 A path must survive its lifecycle 78
20 Measuring the cost of privacy 83
21 Reduce the work without weakening the boundary 88
22 What can be masked at an acceptable cost 93
Appendix A One request through the complete machine 98
Appendix B Reading and extending the evidence 102
References 105

Preface

A computer can display a complete web page while concealing almost everything it did to obtain it. The screen gives little indication that the machine resolved a name, selected an address, chose a route, exchanged several kinds of messages, checked a cryptographic identity, and asked other computers to carry its traffic. When the page appears slowly, those same hidden operations become possible explanations. When privacy matters, they become places where information may escape.

Understanding these operations changes the questions we can ask. An encrypted connection may protect the words in a message while exposing the machines exchanging it. A proxy may replace the address visible to a website while leaving a direct name-resolution query elsewhere. A firewall may enforce the right restrictions during ordinary use but apply them too late during startup. An application may take less time to launch after an optimization even though its network connection has not become faster.

This book follows those distinctions through the machinery that produces them. It begins with communication between computers and works toward a question that requires everything along the way: how can network traffic be masked, and its associated information protected, while preserving performance to the greatest practical extent?

The word "masked" will need careful treatment. We will earn that treatment by first learning what an ordinary connection contains, which components process it, and who can observe each stage. There is no useful shortcut around those foundations. A person who understands how a route, socket, or certificate works can reason about a privacy tool more accurately than someone who has memorized its advertised features.

Ghostnector, my Linux network-privacy project, supplies engineering case studies. Its records contain design decisions, defects, test results, and performance investigations. They also contain corrections to earlier records. These cases are useful because a working implementation must face details that an architectural sketch can leave unresolved. They do not establish universal guarantees for other machines or future versions.

The case studies distinguish source inspection from reported experiments. Some raw logs are present in the supplied repository; others are named in qualification reports but are absent from this source set. When a result rests on a report, the text says so. Test counts describe the documented campaign, rather than tests

newly performed for this book. Hypothetical examples are identified as examples, and their numbers are chosen to explain a mechanism rather than to simulate measurements.

You do not need formal computer science training. You do need to be willing to follow a mechanism across several steps. Technical terms appear where they become useful and receive an explanation there. The later chapters depend on the earlier ones, although readers who already understand a topic can use the chapter headings and references to move ahead.

Sources appear as unobtrusive identifiers such as [R14] and [G01]. The reference section gives their titles and locations. R identifies external technical material; G identifies Ghostnector records. Diagrams are original explanatory drawings based on those mechanisms, rather than reproductions of source illustrations. Their captions state their scope. The source record at the end explains the chronology and the limits of the evidence.

This is the Definitive Edition. Its evidence boundary is the supplied repository at commit 17a03c4 and technical sources consulted on 24 October 2026. That boundary is a way to make the text auditable. It is not a claim that a protocol or a privacy system stops changing when a book describes it.

Definitive Edition. Copyright (c) 2026 Emil Valiyev.
Text and original artwork by Emil Valiyev. AI assisted the final editorial audit of wording, spelling, organization, and structural consistency.

FOUNDATIONS

1 A message needs a path

What the machines agree to do

Suppose a laptop requests a page from a server. A server is a computer, or a program on one, that accepts requests and provides a service. The laptop runs the client program that initiates this exchange. Those are roles in a particular interaction. The same machine can be a client in one conversation and a server in another.

The request begins as data held in memory. Data is represented by bits, values conventionally written as zero and one. A group of eight bits is a byte. Text, images, and instructions all have byte representations, but a receiver needs more than a sequence of changing electrical or radio conditions to interpret them. The machines must agree on how a transmission starts, what its fields mean, how to identify its recipient, and what to do when something goes wrong. A protocol is such an agreement about message formats and behavior.

A computer network connects devices so that they can exchange data under these agreements. The connection can be a cable, optical fiber, a radio link, or several kinds of link joined together. The Internet is an interconnected collection of networks. No single cable, addressing database, or central forwarding machine carries every Internet conversation.

Sharing communication infrastructure makes networks useful. A program can request data stored elsewhere without owning a dedicated physical connection to every possible provider. The price is coordination. Several senders may compete for the same transmission capacity. A destination may be unavailable. Different portions of the route may have different limits. Network protocols turn those conditions into behavior that applications can use, but they cannot eliminate the conditions themselves.

The application does not normally construct every signal that leaves the laptop. It asks the operating system to communicate. The operating system manages the hardware and the resources shared by running programs. Its networking stack is the cooperating implementation of network protocols and related processing. The stack supplies services such as addressing, route selection, and transport delivery. Below it, a driver operates a particular network device.

Bits become signals

A network interface is a machine's attachment to a network. Some interfaces correspond to physical hardware; others exist entirely in software. A network interface controller, often abbreviated NIC, is hardware that sends and receives network traffic. An Ethernet controller can communicate over a cable. A Wi-Fi adapter uses radio and the rules of the relevant wireless network.

Neither device simply emits the characters in a web address. Its physical layer encodes bits into changes in a medium and recovers bits from received signals. A receiver has to synchronize with the transmission and distinguish valid information from noise. The encoding and signaling details depend on the technology. Fiber, twisted-pair copper, and radio do not carry identical physical representations even when they ultimately convey the same application bytes.

The machines also organize those bits into bounded link-layer units called frames. An Ethernet frame includes addressing and type information as well as carried data and error-detection information. The link layer is the set of mechanisms responsible for communication over a particular link. Detecting a damaged frame does not recover the missing application data by itself. Recovery may occur at this layer, at a higher layer, or through an application starting again.

A wired link can connect a laptop to a switch. A switch forwards frames within a local network using link-layer information. A Wi-Fi access point coordinates access to a wireless network and commonly bridges wireless traffic into a wired one. A domestic box may combine an access point, switch, router, and other services in a single enclosure. The enclosure is a product boundary; the functions remain distinct.

On a shared wireless channel, a device may need to wait before transmitting. Nearby users, interference, signal quality, and retransmissions can change how long that wait lasts. A high advertised link rate therefore does not promise equally high application throughput. Throughput means the rate at which useful data is actually delivered under a stated measurement. The distinction will matter when we later compare privacy architectures.

Traveling beyond one link

A local area network, or LAN, connects devices within a limited administrative or physical area. A wide area network joins networks or sites across greater distances. These terms describe scope, rather than one mandatory technology. A local network can use Ethernet and Wi-Fi together. A wide area connection may include fiber, radio, leased infrastructure, or an overlay built on other networks.

To communicate beyond its local link, the laptop sends data toward a router. A router examines network-layer information to decide the next part of a packet's path. A packet is a bounded network-layer unit containing a header and carried data. A header contains fields that the receiving protocol implementation needs to process the unit. The carried data is often called the payload. The same bytes can be payload at one layer and a header at the next layer inside it.

An IP packet, for example, can be carried as the payload of an Ethernet frame. IP is the Internet Protocol, which supplies addressing and packet delivery across connected networks. The router removes the incoming link framing, examines the IP packet, and prepares suitable framing for the next link. The next frame's link-layer addresses are usually different. This does not ordinarily change the packet's ultimate destination address.

The next router repeats the forwarding process. It does not need the whole web request in order to choose a next hop. A hop is one forwarding step along the route. Forwarding decisions can follow different paths in opposite directions, and the path can change. A diagram showing one line between client and server therefore represents a relationship, not necessarily a fixed physical circuit reserved for that exchange.

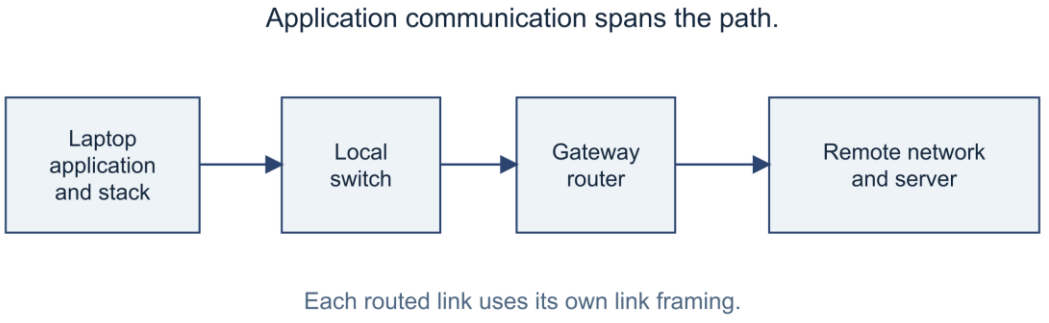


Figure 1.1. A simplified ordinary path. Each link carries its own framing. Application communication spans the entire path; link framing generally does not. This drawing omits access-provider internals and alternate routes. Sources: [R05], [R06], [R07].

Layers describe responsibilities

Layering makes it possible to change one part of communication without redesigning every other part. An application can use the same transport service over Ethernet or Wi-Fi. A router can forward an IP packet without understanding the image format inside it. These separations are useful engineering boundaries, although actual implementations cross them for efficiency or policy.

Encapsulation is the process of carrying one protocol's unit inside another. An application produces bytes. A transport protocol organizes them for communication between applications. IP carries the transport unit across networks. A link protocol carries the IP packet over one link. Decapsulation is the corresponding interpretation and removal of outer structures at the receiver. It need not involve physically copying and stripping a new block of memory at each conceptual boundary.

The Open Systems Interconnection, or OSI, reference model names seven layers: physical, data link, network, transport, session, presentation, and application. It is a framework for discussing responsibilities, not a list of seven processes that every Internet packet must visit. The TCP/IP model, named for the Transmission Control Protocol and Internet Protocol, groups the deployed Internet architecture more directly into link, Internet, transport, and application responsibilities. Some descriptions separate the physical layer as a fifth layer. The naming difference does not change a packet's behavior.

Real implementations resist overly tidy pictures. Encryption may protect application communication, a tunnel, or a link. A virtual interface may hand packets to a program. Hardware may compute checksums or divide a large buffer into transmission units. The right question is what a component actually receives, changes, and emits. Its placement in a familiar diagram is secondary.

Topology describes how the participants are connected. Physical topology concerns cables, radios, and devices. Logical topology concerns the communication relationships created over them. A virtual private network can make two remote sites appear connected through an additional logical interface even though their packets traverse several ordinary Internet networks. We will examine the trust implications of that arrangement later.

Time starts accumulating immediately

Even this first path contains several kinds of delay. A sender spends time preparing data. A link takes time to transmit the bits. Signals take time to propagate through the medium. Routers may queue packets while waiting for a busy output. The receiver spends time processing what arrives. Latency is the

elapsed delay between specified events. Without those events, a latency number is incomplete.

Round-trip time, or RTT, measures a journey to another point and back. It is useful because many protocols must wait for a response before continuing. A page request that requires several sequential exchanges can take much longer than a single RTT. Adding transmission capacity does not eliminate the need for those responses, and a short physical path does not guarantee short queues.

We now have a path between machines, but not yet an explanation of how application bytes enter it. The next stage lies inside the laptop: memory buffers, operating-system decisions, and hardware work that the screen rarely reveals.

2 Inside the host

The application asks the operating system

A running program is a process. The operating system gives it an execution context, memory mappings, and access to resources under specified permissions. Most applications run in user space, where ordinary program instructions cannot directly alter the kernel's protected state. The kernel is the privileged part of the operating system that manages processes, memory, devices, and the services those processes share.

When an application communicates, it commonly uses a socket, an operating-system object through which a program sends or receives data. Through the socket application programming interface, or API, the application obtains a handle and calls operations such as connect, send, and receive. A system call transfers a request from the process into the kernel. This transition is controlled; it does not give the program general authority over the kernel. A library may perform substantial work before making the call, including encrypting application data or buffering small writes. [R13]

For a conventional TCP socket, a successful send usually means the operating system accepted bytes into its outgoing machinery. It does not establish that the remote application read them. TCP, the Transmission Control Protocol, will receive its full treatment later. For now, its relevant service is an ordered stream of bytes between endpoints. The kernel may keep data for retransmission and decide when to send it according to network conditions.

The application therefore sees an interface to communication, not a direct view of the cable. Its own buffer, the socket's queues, the network stack's packet representations, and the device's queues can all contain related data at different moments. When a program reports that it has written a megabyte, the physical link may still be transmitting part of it.

Memory belongs to several views

RAM, or random-access memory, holds data the machine is currently using. The central processing unit, or CPU, executes program instructions. A process normally works with virtual addresses. The processor and operating system translate them into the memory locations that back those addresses, subject to access permissions. Kernel memory holds privileged operating-system structures, including networking state and buffers.

A NIC can often transfer data using Direct Memory Access, or DMA. The driver prepares suitable memory regions and tells the device how to access them. The device then reads or writes those regions without requiring the CPU to copy every byte through explicit instructions. The CPU still configures the operation, manages ownership, and processes the resulting data. DMA changes the transfer mechanism; it does not make the whole networking task independent of the processor.

The addresses used by a device for DMA need not equal an application's virtual addresses or even the CPU's physical addresses. An input-output memory management unit can translate device-visible addresses. The driver must use the operating system's DMA mapping rules to arrange a valid transfer and observe the platform's coherence requirements. Treating all memory addresses as interchangeable conceals a real part of the hardware contract. [R01]

A descriptor is a small record describing a buffer or transfer. Many NICs use descriptor queues arranged as rings. For reception, the driver makes buffers available to the device. The device writes received data and updates completion information. The driver later reclaims or replaces those buffers. For transmission, the driver describes outgoing data and eventually learns that the device has finished with it.

This arrangement explains a basic source of pressure. If the receiver cannot supply and process buffers quickly enough, queues can fill and frames can be lost. More memory may absorb a short burst, but it cannot make a permanently overloaded processor or link service work at an unlimited rate.

Receiving a frame

Consider a valid frame arriving at the laptop's NIC. The hardware receives the signal, checks the relevant link conditions, and places accepted data into an available receive buffer. Depending on the hardware and driver, some validation or classification may happen before software sees the frame. The device notifies the CPU through an interrupt or participates in a polling arrangement.

An interrupt is a notification that requires processor attention. Handling an interrupt for every arriving packet can become expensive at high rates. Linux's NAPI mechanism lets drivers process work through budgeted polling after scheduling it, commonly combining notification with batches of packet processing. Its exact execution context and configuration vary. The important point is that an incoming packet does not necessarily cause one dedicated interrupt followed by one immediate application wakeup. [R02]

The networking stack represents received information using data structures and associated buffers. Linux commonly uses a structure called sk_buff. That structure contains metadata and references to the packet data; it is not simply a fixed box containing every byte. Payload can be stored in several associated memory regions. Sharing or cloning metadata does not necessarily copy the underlying payload. [R03]

The stack interprets the link-layer type, then the network header, then the appropriate higher-level protocol. It checks whether the packet is for this host, should be forwarded, should be rejected, or should be delivered to a local protocol implementation. Firewall hooks may inspect it along the way. If it belongs to an existing TCP connection, the TCP implementation validates sequence information, updates its connection state, and makes newly usable bytes available to the receiving socket.

The application may be asleep while waiting for data. The kernel can wake it, but the scheduler still decides when it runs. The scheduler is the operating-system component that assigns processor time to runnable tasks. A context switch changes the running execution context. A syscall does not always require switching to a different process, and a packet does not always require an immediate context switch. Those distinctions matter when attributing performance costs.

When the application finally reads, ordinary socket APIs may copy data into its user-space buffer. Other interfaces can avoid some copies or share memory under additional constraints. A useful explanation must say which path is under discussion. "Zero copy" is an optimization claim about specified transfers, not proof that no copying or bookkeeping occurs anywhere in the communication.

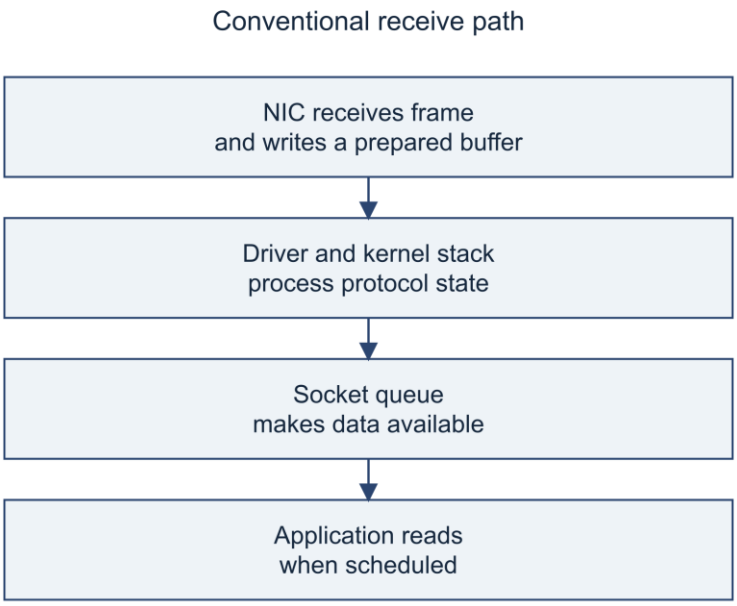


Figure 2.1. A conventional receive path. DMA can transfer data into prepared buffers; software then processes protocol state and makes data available through a socket. Driver and hardware details vary. Sources: [R01], [R02], [R03], [R13].

Sending works through queues too

For outgoing data, the stack selects protocol behavior, determines a route, and prepares a representation the driver can transmit. A queueing discipline can decide when packets become eligible for the device. The driver maps or references suitable buffers and submits transmit descriptors. The NIC reads those buffers and produces the link transmission.

Modern systems may defer some work to hardware. Checksum offload lets a NIC calculate specified error-detection fields. Segmentation offload lets software hand over a larger unit that the device divides into packets suitable for the link. On reception, aggregation can combine related incoming work before upper layers process it. These optimizations reduce work per delivered byte, but they also complicate packet capture: a capture inside the host can show units or checksums that differ from the final wire representation. [R46]

Encapsulation therefore describes the logical protocol structures, not a requirement that the kernel repeatedly allocates and copies a completely new object. An efficient implementation can reserve header space, reference shared pages, or adjust offsets. It still has to preserve the protocol's visible behavior. Optimization is useful precisely because the same behavior can sometimes be implemented with less internal work.

Several CPU cores can process network activity. Receive Side Scaling, or RSS, lets suitable NICs distribute flows among receive queues. Linux also has software steering mechanisms. Distribution can improve capacity, but sending related work across cores has cache and coordination costs. A cache is a small, fast memory used to keep recently accessed information close to a processor. Moving work can lose that locality. More parallelism is a choice with conditions, rather than a universal improvement. [R04]

Ports do not point into RAM

A TCP or UDP port is a numerical field used by transport protocols. UDP, the User Datagram Protocol, sends bounded messages without TCP's ordered-stream service. When an incoming packet reaches either transport implementation, the operating system uses addresses, protocol, port numbers, binding rules, and, where relevant, connection state to select a socket.

The port number is not a RAM address. A packet containing destination port 443 does not instruct the NIC to write into memory location 443. The NIC and driver first arrange the receive transfer using their configured buffers. Later processing interprets the packet and determines which socket should receive its data. The socket refers to operating-system state, and the application accesses that state through its handle.

Nor does a port identify one program everywhere. Different machines can use the same number. One host can use the same local port in different connections, subject to protocol and binding rules. A listening server socket and an established connection are related objects with different matching behavior. We will return to those rules after learning the addresses that participate in the match.

This host-level journey already separates several possible bottlenecks. The application may be slow to produce data, the kernel may be busy, a device queue may be full, or the receiving process may not be scheduled promptly. A privacy system can add another program and another set of buffers to this path. Before assigning a cost to encryption or anonymity, we have to establish where the machine actually spends its time.

3 Addresses and neighbors

Two questions for one transmission

The laptop needs to distinguish the destination it wants to reach from the next device that will carry its packet. Those are different questions whenever the destination is beyond the local link. IP addresses identify network-layer endpoints and interfaces in the context of routing. Link-layer addresses identify participants in delivery over a particular link.

On an ordinary Ethernet link, a Media Access Control address, or MAC address, appears in the frame. It is commonly 48 bits long and written as six hexadecimal groups. Hexadecimal is base sixteen notation, which makes binary values compact to display. MAC addresses are used for link delivery and switch learning. They are not permanent proof of a person's identity. Software can change them, and some devices use randomized addresses under specified conditions.

A switch learns which source MAC addresses appear on its ports and uses that information when forwarding later frames. If it does not know the destination, it may flood the frame within the relevant broadcast domain. A broadcast domain is the set of link participants that receive a link broadcast. A virtual LAN, or VLAN, can divide switching infrastructure into separate logical domains. Knowing that two devices connect to the same physical switch does not prove they belong to the same logical network.

Wi-Fi also has link-layer addressing, but its frame formats and access procedures differ from Ethernet's. A wireless transmission may involve an access point and several address fields describing different roles. It is therefore misleading to assume that every local frame is an Ethernet frame with a radio substituted for the cable. The common purpose is delivery over a link; the mechanism depends on that link.

IPv4 and the prefix

An IPv4 address contains 32 bits, normally displayed as four decimal numbers. In the illustrative address 192.168.10.23, each number represents one byte. An IPv4 prefix identifies a leading sequence of address bits. The notation 192.168.10.0/24 means that the first 24 bits form the prefix. It does not mean that the final number is a device count.

Suppose the laptop has 192.168.10.23/24 and a directly connected route for 192.168.10.0/24. A destination of 192.168.10.60 matches that route. A destination of 198.51.100.40 does not. The latter address belongs to a documentation range here; it does not identify a real server in this example. The host compares destination bits with its routing entries and selects an applicable route. That decision determines whether it needs a local neighbor or a gateway.

For a /24 prefix, 256 bit combinations share those leading bits. Conventional IPv4 subnet use reserves particular addresses for the subnet and broadcast, but the often repeated rule "subtract two from every subnet" has exceptions, including point-to-point /31 use. Subnetting is the division of an address range into smaller prefixes. Its mechanism is a change in how many leading bits the routes treat as the network prefix, rather than a change in the physical meaning of an address. [R06, R50]

An address also has a scope and an allocation context. Private IPv4 ranges can be reused inside unrelated networks. They cannot be assumed globally unique or directly reachable from the Internet. A public address may be shared by many users through address translation. An address observed in a log therefore identifies a network relationship at a time, not automatically one individual or one physical computer.

Finding the local recipient

After selecting an IPv4 next hop on Ethernet, the host needs its link-layer address. The Address Resolution Protocol, or ARP, supplies that mapping. If the host lacks a usable cached entry, it broadcasts a request asking which participant has the target IPv4 address. A respondent supplies the corresponding link address, and the sender stores the result for subsequent traffic. [R05]

If the remote server lies beyond the local subnet, the ARP target is ordinarily the gateway's local address. The IP packet still names the remote server as destination. The Ethernet frame names the gateway as the immediate link recipient. Confusing those fields leads to the mistaken belief that every router is the final destination of the packet it forwards.

Neighbor caches reduce repeated discovery overhead. They also introduce state that can expire, become stale, or be manipulated. The initial request may wait for address resolution; later requests may not. A performance experiment that compares one cold neighbor lookup with one warm cache can attribute the difference to the wrong component.

Classic ARP provides no cryptographic proof that a reply is truthful. A hostile participant with suitable link access can try to replace a legitimate mapping. We will examine that threat after establishing the normal forwarding path. At this stage, the important dependency is that IP route selection and link-neighbor discovery solve different parts of one transmission.

IPv6 changes the details

IPv6 addresses contain 128 bits. Their hexadecimal notation can compress runs of zero groups, so the written length varies. A prefix such as 2001:db8:10::/64 identifies the first 64 bits; 2001:db8::/32 is reserved for documentation, so these are again examples. More address space changes allocation possibilities, but it does not remove the need for route selection, neighbor discovery, or access policy. [R07]

IPv6 uses Neighbor Discovery rather than ARP. Neighbor Discovery operates through ICMPv6, the IPv6 control-message protocol, and supports functions including link-address resolution, router discovery, and reachability tracking. Router advertisements can help hosts learn prefixes and default routers. Stateless address autoconfiguration can let a host form addresses using advertised information and local rules. DHCPv6 is another configuration mechanism with a different role; it is not simply the IPv4 exchange with longer addresses. [R08]

A host may have several IPv6 addresses on one interface, including a link-local address used within the local link. Temporary addresses can reduce some forms of long-term address tracking, but they do not conceal the host's entire prefix, traffic behavior, or application identity. A privacy claim must say which observation the address change is intended to frustrate.

IPv6 also exposes the cost of careless filtering. Blocking all ICMPv6 can interfere with basic network operation, including discovery and packet-size handling. A system deliberately disabling IPv6 as part of a narrow confinement policy has made a compatibility choice. A system intending to support IPv6 must handle its required control traffic rather than borrowing an IPv4 rule without examining the consequences.

Interfaces can multiply identities

A laptop can have a Wi-Fi interface, an Ethernet interface, a loopback interface, and several virtual interfaces at the same time. Loopback delivers communication within the host. A virtual interface can represent a software link, tunnel, or connection between isolated networking environments. The operating system's routes decide how destination traffic uses these interfaces.

Multiple addresses create several possible paths. A policy that redirects IPv4 traffic but leaves a usable IPv6 route may permit an application to reach the destination through IPv6. A rule attached to one interface may cease to constrain traffic after the machine changes networks. A configuration file that lists one resolver may be superseded when another interface becomes active.

Those are examples of how an incomplete path model creates exposure. The machine has not somehow ignored a privacy setting; it has used a path that the setting never controlled. We can only evaluate that behavior after identifying the interfaces, addresses, routes, and protocol families involved.

The same distinction keeps measurements honest. Wireless retransmissions belong to a local link. A neighbor-cache miss belongs to next-hop discovery. A longer Internet route belongs to forwarding beyond that link. They can all delay one page load, but they require different explanations and different interventions.

4 Routes and rewritten addresses

Choosing a next hop

A routing table records how a host or router can forward traffic toward destination prefixes. An entry can specify an output interface, a next-hop gateway, and other selection information. When several prefixes match, ordinary IP forwarding favors the most specific matching prefix, the one with the greatest prefix length. Other policy rules and route attributes can affect the complete lookup, so a single table is sometimes only part of the story.

Consider three illustrative entries: a connected route for 192.168.10.0/24, a route for 203.0.113.0/24 through a specialist gateway, and a default route through the ordinary gateway. The default route matches destinations not covered by a more specific choice. It is often written 0.0.0.0/0 in IPv4 or ::/0 in IPv6. A /0 prefix contains no fixed leading bits and therefore matches every destination in that family.

A gateway is the next device used to reach a network beyond the local link. The word can also describe a translator between services, so the context matters. A default gateway does not need to be the destination's owner or know the full application request. It needs a suitable next-hop decision of its own.

Routing has a control plane and a data plane. The control plane establishes and changes forwarding information, through configuration or routing protocols. The data plane applies that information to traffic. A correct data-plane lookup cannot rescue a control plane that supplied the wrong route. Conversely, correct routing information does not establish that every packet-processing rule allows the resulting path.

Internet networks are administered by different organizations. An autonomous system, or AS, is a routing domain identified in interdomain routing. The Border Gateway Protocol, BGP, lets these systems exchange reachability information and

apply routing policy. Its selected route need not be the physically shortest route or the route with the lowest measured delay. Commercial and administrative policy also matters. [R26]

A packet changes on the way

Routers normally decrement a packet's remaining hop allowance: Time to Live in IPv4, or Hop Limit in IPv6. Despite the IPv4 name, ordinary forwarding treats this as a hop count. A packet that exhausts it is discarded rather than circulating indefinitely. ICMP messages can report such conditions, which is part of how diagnostic tools infer a path.

ICMP, the Internet Control Message Protocol, carries network control and error information. It is not a universal guarantee that every failure produces a usable explanation. Firewalls, rate limits, and routing conditions can prevent the messages from arriving. A silent timeout therefore does not locate the failure by itself.

Each link has a maximum transmission unit, or MTU, limiting the size of the network-layer packet it can carry in one link unit. Encapsulation adds headers and can reduce the space available to an inner packet. Sending packets larger than an effective path supports can trigger fragmentation or failure, depending on protocol and settings. IPv6 routers do not perform the same in-transit fragmentation as IPv4 routers. Hosts must handle packet-size constraints with the appropriate discovery or adaptation mechanisms. [R06, R07]

This is why a tunnel can appear connected while some larger transfers stall. Small handshake messages fit; later packets do not. If the signaling needed to discover a smaller path MTU is blocked, the sender may continue making an unsuitable assumption. Packet-size management is therefore both a performance concern and a compatibility requirement.

Network address translation

Network Address Translation, or NAT, rewrites address information as traffic crosses a boundary. A common home-router form rewrites a private source address and often the source transport port into an externally usable address and port. Return traffic is translated back using maintained state. The combination lets several internal connections share an external IPv4 address.

Imagine two internal clients opening TCP connections to the same server port. The router can assign different external source ports, allowing replies to be associated with the correct internal connection. The translation state relates the original and translated tuples. A tuple here is an ordered collection of fields, such

as source address, source port, destination address, destination port, and protocol.

Source NAT changes the source; destination NAT, often abbreviated DNAT, changes the destination. Linux masquerading is a form of source NAT that uses the outgoing interface's address. A destination rewrite can redirect a connection to a local proxy or another internal server. The application may believe it connected to one address while a kernel rule directs the actual traffic elsewhere. That apparent transparency requires the proxy to recover the intended destination by a mechanism appropriate to the system.

Connection tracking, commonly called conntrack in Linux discussions, maintains state about observed flows. It supports stateful filtering and the association needed for NAT. A tracked UDP exchange is still not a TCP connection: the kernel infers useful flow state from datagrams and timeouts without adding a UDP handshake to the protocol.

NAT is not encryption. An observer after translation can read unencrypted carried content. NAT also does not authenticate an internal client to a destination. Some deployed NAT arrangements obstruct unsolicited inbound traffic because no mapping exists, but that behavior should be distinguished from an explicit filtering policy. The security question is which traffic the device accepts and forwards under which conditions.

The order of decisions matters

Packet processing is a sequence of operations. A destination rewrite, a route lookup, a filter verdict, and a later route adjustment may see different versions of the same traffic. A rule that asks about an output interface before rerouting does not necessarily observe the interface that will finally transmit the packet.

Linux's Netfilter hooks let components such as the nftables firewall framework participate at defined points. Locally generated traffic takes the output path; traffic forwarded for another host takes a forwarding path. Chains attached to a hook have priorities that influence processing order. Identical priorities do not express a dependable ordering relationship between separate base chains. [R28]

Ghostnector's early release-candidate campaign exposed this distinction. Its records attribute two falsified claims to NAT and filter chains sharing a priority, and to an assumption that a redirected packet's output-interface match already described its eventual loopback delivery. The documented correction separated priorities and matched the rewritten loopback destination appropriately. These were historical defects; later records describe fixes and regression coverage. [G02, G13, G14]

The lesson extends beyond that implementation. A diagram saying "redirect, then allow the proxy" does not establish which packet fields a real kernel exposes at each stage. The mechanism has to be tested against the actual path. This is one reason a configuration that looks plausible can fail when placed in a different namespace or combined with another policy.

Routes can fail safely or unsafely

Removing a route can stop traffic, but a newly available default route may restore it. A VPN that installs a preferred route may leave an ordinary route available when the tunnel disappears. Whether traffic then uses that route depends on the rest of the policy. "The tunnel stopped" and "the machine cannot send directly" are different statements.

The reverse problem is also possible. A restrictive policy can remain after the service that made its permitted path useful has stopped. The result is intentional or accidental loss of connectivity. Availability means that the required service can be used; confinement means that prohibited traffic cannot take an alternative path. A system can satisfy confinement while failing availability.

Routing determines possible next steps. Translation changes the addresses used in those steps. Filtering determines which steps are permitted. A complete privacy architecture must account for all three, including transitions between configurations.

5 Getting an address and finding a name

Configuration arrives over the network

A newly connected machine needs usable configuration before it can make the ordinary request we have been following. That may include an address, a prefix, a default route, and resolver information. An administrator can enter these values manually, but many networks distribute them automatically.

For IPv4, the Dynamic Host Configuration Protocol, DHCP, coordinates the process. In a common initial exchange, a client announces that it needs configuration, a server offers a lease, the client requests an offered configuration, and the server acknowledges it. A lease grants use under conditions and for a limited time. The client later renews rather than treating the address as permanent. Relays can carry DHCP exchanges between clients and a server elsewhere in the network. [R09]

Because the client may not yet have ordinary IP configuration, initial exchanges use special addressing behavior. DHCP uses UDP; client and server use different port roles. A firewall exception must match the actual direction the client sends, rather than the direction the server uses for a reply. Permitting the wrong destination port can leave initial tests apparently healthy while renewal later fails.

Ghostnector's historical DHCP defect illustrates that error. The project records describe an exception written for UDP destination port 68, although the relevant client transmission uses source port 68 and destination port 67. The correction added a source-port match and a corresponding invariant in the policy representation. This is an example of a documented project defect, not a claim that every DHCP implementation uses the same surrounding process model. [G02, G13]

Automatic configuration also establishes a trust relationship. A network-supplied resolver can influence which answers the machine receives. A supplied gateway influences where traffic goes. An untrusted access network therefore has opportunities to redirect or observe traffic even before a browser asks for a page. Later cryptographic authentication can constrain what a redirected peer can successfully impersonate, but it does not make the initial configuration source trustworthy.

DNS is a distributed system

People commonly use names such as www.example.com, while IP forwarding requires addresses. The Domain Name System, DNS, stores typed records associated with names in a hierarchical namespace. It is used for much more than one name-to-address operation. An A record holds an IPv4 address, an AAAA record holds an IPv6 address, and other types describe aliases, mail routing, and further information. [R10, R11]

An application often asks a local resolver interface to obtain an answer. A small client resolver, or stub resolver, can forward a request to a recursive resolver. The recursive resolver performs the work needed to find an answer, consulting caches and authoritative sources. An authoritative server answers for the zones it serves. A zone is an administratively managed portion of the DNS namespace; it is related to, but not identical with, the everyday notion of a domain name.

For an uncached lookup, the recursive resolver can follow referrals through the hierarchy: a root server directs it toward the relevant top-level domain, that infrastructure directs it toward the appropriate authoritative servers, and the authoritative response supplies the required record or another step such as an alias. Actual lookup behavior depends on cached delegations, query minimization, and resolver implementation. A new visit does not imply a completely fresh trip through every level.

Records have a time to live, or TTL, used to govern caching. This DNS TTL concerns record reuse, not the IP hop limit discussed earlier. Reusing the same abbreviation does not make the mechanisms related. Cached positive and negative answers reduce repeated work but can also preserve a result after an administrator changes the underlying data.

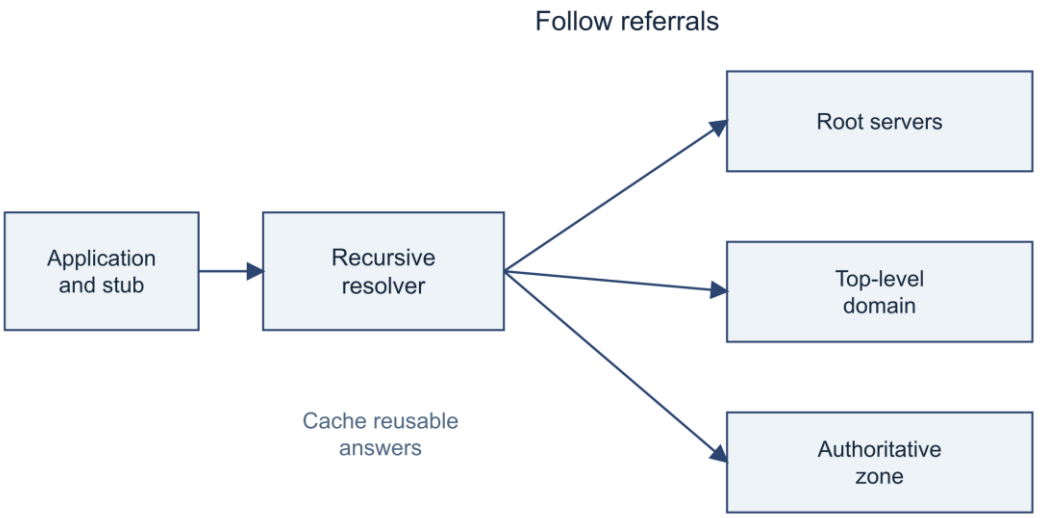


Figure 5.1. An illustrative uncached recursive lookup. The application asks a recursive resolver, which follows referrals. Cache hits can omit several exchanges. Sources: [R10], [R11].

A name can take several paths

The operating system's resolver is not necessarily the only resolver a program uses. A browser can maintain its own DNS cache or use encrypted DNS independently. A proxy-aware application can pass a hostname to a proxy, letting the proxy resolve it remotely. Another application may resolve locally and pass only an IP address.

Those choices change both observation and measurement. Local resolution can expose a name to the local resolver even if the later TCP connection uses a proxy. Remote resolution moves that visibility to another place. An encrypted channel to a recursive resolver conceals the query from some observers between client and resolver, while the resolver still receives the question.

A DNS leak is a lookup that crosses a boundary contrary to the intended privacy policy. It is not defined merely by whether port 53 appears in a capture. The policy may require every lookup to follow a protected path, or it may deliberately allow a specified encrypted resolver. The observer, destination, path, and promise must be named before the term "leak" has a precise meaning.

Likewise, changing a resolver configuration file does not enforce all future behavior. A program can send queries to a hard-coded address. Another component can replace the file after a network change. Kernel rules can redirect conventional DNS traffic or deny it, but encrypted DNS over an allowed application connection requires a different analysis. A port-based rule does not identify every DNS message inside an encrypted stream.

Resolution costs are conditional

A DNS lookup can finish quickly from a local cache, take one network round trip to a nearby recursive resolver, or wait through several remote requests and retries. Measuring one of these states and applying the result to all name resolution is an error. Even successive lookups for the same name can differ if the application, operating system, and recursive resolver have different caches.

An encrypted resolver connection adds connection establishment and cryptographic processing when it is new. A reused connection can amortize those costs across many queries. Amortization means distributing a fixed cost over several operations. It does not make the initial cost disappear, and the first request after startup can still experience it.

Negative responses matter too. A missing name can be a valid DNS answer. A timeout may instead mean packet loss, an unreachable resolver, a broken protected path, or a policy refusing the query. A diagnostic that reports every failure as "DNS is down" has discarded distinctions the reader needs in order to repair the problem.

For privacy measurements, the resolution path has to match across compared configurations. Passing a hostname through SOCKS and sending a local DNS query followed by a transparent TCP connection are different workloads. A difference between them cannot be assigned entirely to the transparent forwarding software. Ghostnector's later performance campaign explicitly identified this baseline problem, which we will examine with the actual records. [G07]

We can now name a remote service and find possible addresses for it. The next question is how the operating system distinguishes simultaneous conversations between programs, and what kind of delivery those programs can request.

6 Sockets ports and datagrams

An address does not select an application

An IP address gets a packet to the relevant network-layer endpoint. The host still needs to decide which local communication object should receive the carried data. Transport protocols provide additional fields for that decision. TCP and UDP both include source and destination port numbers, each represented by 16 bits. The same numerical port in TCP and UDP belongs to different transport namespaces.

A server commonly binds a socket to a local address and port. Binding asks the operating system to associate the socket with that local endpoint under the applicable rules. A wildcard bind can accept traffic addressed to several local interfaces. A loopback-only bind limits ordinary reachability to local communication in that networking environment. These are materially different exposure choices.

A TCP server then listens for connection attempts and accepts established connections. The listening socket can remain available while accepted sockets represent individual conversations. Clients commonly obtain temporary local ports chosen by the operating system. Such a port is often called ephemeral because it is allocated for the communication rather than functioning as a stable service identifier.

An established TCP connection is normally distinguished by its protocol and local and remote address-port pairs within the relevant networking context. Two clients can therefore connect to the same server port simultaneously. Even one client can maintain several connections to it using distinct local endpoints. The kernel does not need a unique server port for every conversation.

UDP delivery has its own matching rules. A socket can receive datagrams from multiple peers, or it can be associated with a selected peer through connect.

Connecting a UDP socket does not perform TCP's handshake. It can set the default destination, constrain which peer traffic is delivered to that socket, and affect error handling. Reuse options, address binding, and operating-system behavior complicate the details, which is why "one port equals one process" is an unreliable rule. [R12, R13]

What UDP provides

UDP carries bounded messages called datagrams. Its header supplies port numbers, length, and a checksum field. The application chooses a datagram's contents. UDP does not itself require the receiver to acknowledge delivery, reconstruct an ordered byte stream, retransmit lost data, or negotiate a connection before sending. A datagram can arrive late, arrive more than once, arrive out of order, or fail to arrive.

Those omissions are a service model. They let an application decide whether a late message is still useful, whether to resend it, and how to react to congestion. An application that needs reliable delivery over UDP must supply the necessary additional mechanisms. QUIC, which we will examine shortly, does exactly that. Using UDP underneath a protocol does not tell us that the complete protocol is unreliable or unencrypted.

A datagram boundary is visible to a receiving UDP application. If the application submits two messages, the receiver does not obtain them as an arbitrary concatenated stream. It may lose one, receive them separately, or receive them in a different order. A receive buffer too small for a datagram can cause truncation under the API's rules. A larger network datagram can also require IP fragmentation, increasing the consequences of a fragment being lost.

The checksum detects some accidental corruption; it is not cryptographic authentication. An attacker who can construct a replacement packet can calculate an ordinary checksum for it. IPv4 UDP allows an absent checksum in its base protocol, while IPv6 generally requires one, with narrowly defined exceptions in other specifications. Neither fact means an ordinary checksum protects against deliberate forgery. [R12, R07]

Buffers change what the application observes

A socket receive buffer holds incoming data until the application consumes it. If datagrams arrive faster than the process reads them, the queue can fill and later datagrams can be dropped. Raising the buffer limit may tolerate a burst, but it cannot permanently compensate for an application that consumes data too slowly.

Outgoing buffers matter too. A successful UDP send does not prove that the remote application received the message. It may only establish that the local system accepted the operation. Some errors can be delivered later through operating-system mechanisms, while others never return useful feedback. The application must define what counts as success at its own level.

For a voice call, receiving the next short piece of audio promptly may matter more than recovering a previous piece after its playback deadline. For a file transfer, losing a piece is unacceptable unless another mechanism retrieves it. Those requirements help explain different transport choices. They do not justify sending unlimited traffic without congestion response. A UDP application still shares links and queues with other users.

Port conventions help programs locate services, but a port number does not establish the payload's meaning. Traffic on port 443 might carry a web protocol, another permitted service, or deliberately misleading data. A firewall allowing a port is applying a reachability rule, not necessarily validating an application protocol. Encryption can further prevent an intermediate device from seeing the carried protocol details.

Local proxies are network services too

A proxy listening on a local socket can become reachable by a wider set of programs than intended. Loopback restricts which network paths ordinarily reach it, but it does not distinguish every local user. A Unix-domain socket, which communicates through the host's local socket facility rather than ordinary IP routing, can use filesystem permissions and peer credentials to impose a different boundary.

Ghostnector separates control requests from carried traffic. Its helpers receive constrained local requests rather than arbitrary commands or firewall text supplied by an application. Its APP relay listens inside a particular namespace and refuses connections for which it cannot establish an intercepted original destination. The source describes that refusal as part of the relay contract, reducing the chance that the listener becomes a general proxy for caller-chosen destinations. [G04, G11]

This is a useful distinction between an interface and a privilege. A program may need access to a proxy to move application data, but that does not mean it should be able to reconfigure the system's networking policy. The sockets, permissions, and message formats should reflect the difference.

TCP adds mechanisms to create a different service: a reliable ordered stream, provided the connection can continue making progress.

7 TCP keeps a conversation in order

Establishing connection state

TCP presents applications with a stream of bytes. It does not preserve the boundaries between individual application writes. If a sender writes 100 bytes twice, a receiver might read 200 bytes at once, two reads of 100, or another division. The application protocol must define how to identify complete messages in that stream.

Before an ordinary TCP connection carries application data, the endpoints establish state. In the familiar three-way handshake, the initiator sends a segment with the SYN flag, the receiver replies with SYN and ACK, and the initiator acknowledges the reply. A flag is a control bit in the TCP header. SYN synchronizes initial sequence information; ACK indicates that the acknowledgment field is significant. Options can negotiate capabilities and limits during this exchange. [R14]

Each side chooses an initial sequence number. Sequence numbers identify positions in the byte stream, allowing the receiver to distinguish new data, missing data, and duplicates. The handshake also demonstrates that the endpoints can exchange traffic using the proposed addressing relationship. It does not cryptographically establish the server's identity. An attacker able to satisfy or interfere with the network exchange is a separate security problem.

The connection's state exists in both kernels. Routers along the path need not implement that endpoint state in order to forward the packets, although firewalls and NAT devices often maintain related flow state. Keeping these layers separate helps explain why an intermediate device can forget a translation while the applications still believe their TCP connection exists.

Reliability refers to the byte stream

The sender divides data into segments suitable for transmission. A segment is a TCP protocol unit containing a TCP header and any carried bytes. Each data segment identifies where its bytes belong in the stream. The receiver acknowledges progress, often cumulatively: its acknowledgment can identify the next byte it expects in order.

Take an illustrative stream in which the next expected byte is numbered 1001. If the receiver obtains the next 1,000 bytes without a gap, it can acknowledge 2001. If a later segment arrives while bytes 1001 through 1500 are missing, the receiver cannot present a contiguous stream beyond that gap merely because later bytes are already in memory. Selective acknowledgment, when negotiated, can supply additional information about blocks received out of order.

An acknowledgment means that the receiving TCP implementation has accepted the relevant data according to the protocol. It does not mean the receiving application displayed it, stored it durably, or acted on it. An application requiring confirmation of a database transaction needs its own response, even if TCP has already acknowledged every request byte.

If the sender lacks evidence of successful delivery, it can retransmit. Retransmission timers use observations and rules intended to distinguish reasonable waiting from failure; duplicate acknowledgments and other feedback can support earlier loss recovery. Reordering can complicate the inference. Loss recovery is therefore a stateful decision process, rather than a simple rule that every missing acknowledgment causes an immediate resend.

TCP's promise is conditional. It provides ordered delivery or eventually reports failure; it cannot guarantee progress through a permanently broken path. A connection can remain stalled while timers, application deadlines, and operating-system limits determine when the participants give up.

The receiver and the network have different limits

Flow control protects the receiver. It advertises how much additional data it can accept, based on its buffering and consumption. If an application stops reading, the receiver can reduce the advertised window. A sender obeying it may have to wait even when the physical network is almost empty.

Congestion control responds to the network's ability to carry traffic. A sender maintains a congestion window or equivalent sending allowance, reacts to loss or explicit congestion signals, and attempts to avoid persistent overload. The effective amount of outstanding data is constrained by both the receiver's allowance and the congestion-control state. [R15]

Classic TCP descriptions introduce slow start, which grows the sending allowance as acknowledgments return, and congestion avoidance, which changes its growth after other conditions apply. Modern implementations use different congestion-control algorithms, so one growth equation should not be presented as the behavior of every TCP connection. The shared problem is learning a useful sending rate while responding to a changing path.

An acknowledgment travels back before the sender can use it to guide further transmission. A longer RTT slows that feedback loop. A high-capacity path can therefore require a substantial amount of data in flight to stay busy. Increasing bandwidth alone will not help a sender whose allowed outstanding data is too small for the path's delay.

A hypothetical 100 megabit-per-second path with an RTT of 0.1 seconds has a bandwidth-delay product of 10 megabits, or about 1.25 megabytes using decimal units. That product estimates the amount of data corresponding to one round trip at the stated rate. It is an explanatory calculation, not a measurement of a real service. Actual throughput also depends on congestion behavior, receiver limits, loss, and competing traffic.

Queues can improve throughput and worsen delay

Buffers absorb short mismatches between arrival and service rates. A router can hold a burst instead of immediately dropping it. But a persistently full queue adds waiting time to every packet behind it. A large download can then make an unrelated interactive request feel slow even when the link is continuously busy.

Active queue management attempts to signal congestion before a queue remains excessively full, using marking or dropping according to the mechanism. It differs from simply increasing the buffer. Fair scheduling can also change how competing flows share waiting time. The choice of mechanism depends on where the bottleneck is and what the operator controls. [R45]

Jitter is variation in delay. For an interactive application, a stable 80 milliseconds may be easier to handle than delay repeatedly varying between 20 and 200 milliseconds. A privacy overlay introduces additional queues and feedback loops, so reporting only the median can hide the behavior that users notice most.

Closing and interruption

TCP can close each direction of the stream separately using FIN exchanges. One application may finish sending while still receiving a response. An abrupt reset terminates the connection differently, potentially discarding work the application expected to complete. TIME_WAIT retains state after particular close sequences

to handle delayed traffic and the protocol's reuse concerns. It is purposeful state, rather than a connection that necessarily failed to clean up.

A network change can invalidate addresses or intermediate mappings. A stopped proxy can leave an application's local connection accepted but unable to reach the remote service. The exact point at which a proxy acknowledges a local connection therefore matters to measurement. Timing only the local connect can omit remote setup that happens afterward.

TCP also imposes ordering across the bytes of one connection. If several independent application streams share that connection, a gap in the TCP byte sequence can delay delivery needed by all of them. This is transport-level head-of-line blocking. Application multiplexing can improve concurrency without removing this underlying dependency.

8 The application shapes the traffic

HTTP supplies meaning

The Hypertext Transfer Protocol, HTTP, defines requests and responses used by the web and many other services. A request identifies an operation and a target resource; a response supplies a status and may carry a representation of that resource. Headers provide additional information. HTTP semantics describe what these messages mean, while different HTTP versions define different ways of carrying them. [R16]

A browser requesting a page may need much more than one response. The document can refer to stylesheets, scripts, images, and further services. Some requests depend on information in earlier responses. Others can run concurrently. A page-load time therefore includes application dependencies, processing, and rendering as well as network transfer.

HTTP/1.1 commonly carries textual start lines and headers followed by bodies whose boundaries follow the protocol's framing rules. Persistent connections let several requests reuse a transport connection. Reuse avoids paying a fresh TCP handshake for every resource. It can also preserve connection state that changes later observations, making a warm session different from a cold one.

HTTPS means HTTP carried with the required security association, conventionally using TLS. Transport Layer Security, TLS, is a protocol for authenticating peers and protecting exchanged data against specified forms of reading and alteration. It can protect protocols other than HTTP, and HTTP can operate in contexts other than HTTPS. The two names describe different responsibilities. Chapter 11 explains the security mechanism.

Multiplexing changes the waiting

HTTP/2 represents communication as frames associated with streams over a connection. A stream is a logically separate exchange within that multiplexed communication. Frames from different streams can be interleaved, avoiding some application-level serialization. Header compression reduces repeated information, and flow-control mechanisms constrain carried data. [R17]

Over TCP, however, the receiver still sees one ordered transport byte stream. If TCP lacks bytes from a lost segment, it cannot simply skip the gap to deliver later bytes belonging to another HTTP/2 stream. HTTP/2 improves one layer's organization while retaining the transport's ordering dependency. That distinction explains why multiplexing does not make packet loss irrelevant.

QUIC places a secure transport protocol over UDP. It supplies connection establishment, reliable streams, flow control, loss recovery, congestion control, and other behavior above UDP's datagram service. TLS participates in key establishment, but QUIC carries handshake information and protects its own packets using the integration defined for it. It is inaccurate to describe QUIC as ordinary TCP with a different port or as TLS records placed unchanged into UDP. [R18]

QUIC can deliver usable data from one stream while another stream has a missing range, subject to each stream's own ordering and the protocol's shared constraints. This removes TCP's single-byte-stream ordering dependency between those streams. It does not give them independent physical capacity: they still share congestion control, the path, and implementation resources. A congested link can delay all of them.

Connection identifiers can let QUIC associate traffic with a connection when network addressing changes. Migration requires protocol validation and policy support; a changed address does not automatically establish a safe new path. The identifiers also create observable information whose privacy implications depend on their use and rotation.

HTTP/3 maps HTTP onto QUIC. Its performance depends on the deployment, path, loss conditions, and connection state. A new version number does not establish that every request is faster. Some networks block UDP, and applications may fall back to another supported HTTP transport. If a confinement architecture permits only TCP through its protected path, the availability and timing of that fallback become part of the user experience. [R19]

Setup can dominate a small request

A small response can spend more time waiting for setup than transmitting its body. Name resolution, TCP or QUIC establishment, cryptographic negotiation, and the application request can form a sequence of dependent exchanges. Some can overlap; some require a previous answer. Reusing a connection changes the sequence substantially.

Resumption lets a peer use suitable previous cryptographic state to reduce work in a later connection. Early data, often called 0-RTT data, can reduce waiting under defined conditions, but replay concerns limit which operations are safe to send that way. It is not a general instruction to execute every authenticated request before fresh handshake completion. We will return to replay in the cryptography chapter.

Caching can avoid requesting data at all. A cache stores a result for later reuse under the application's rules. Comparing a cached page with an uncached page measures a different workload, even when both appear in the same browser window. A benchmark should state what was reused, including DNS records, application responses, transport connections, and privacy circuits.

Real-time communication has deadlines

A live conversation has different demands from a document transfer. The receiver needs audio and video data close to their playback times. It may use a jitter buffer to smooth variation in arrival time, holding a small amount before playback. A larger buffer can tolerate more variation while adding delay to the conversation.

The Real-time Transport Protocol, RTP, supplies information such as sequence numbers, timestamps, and payload identification for real-time media. Its companion control protocol, RTCP, conveys reporting and control information, including feedback about reception. RTP does not by itself promise timely delivery, reserved bandwidth, or encryption. Secure RTP adds security mechanisms under a separate specification. [R20]

An RTP timestamp refers to a media clock and lets the receiver relate samples to playback, rather than simply identifying the wall-clock time a packet crossed a router. Sequence information helps identify missing or reordered packets. A receiver can conceal some loss or adapt playback, while feedback can help a sender alter its behavior. Those application decisions determine whether retransmitting late data is worthwhile.

A privacy network designed around low-latency TCP streams cannot automatically carry arbitrary UDP applications with their existing semantics. A

tool may refuse such traffic, require an application-specific gateway, or use a different architecture. Each choice changes compatibility and possibly the threat model. Calling the result "all traffic protected" without naming that boundary leaves readers unable to predict what will work.

Packet boundaries do not reveal application boundaries

One application message may span several transport units. Several messages may share one unit. TLS can divide plaintext into records that do not match either application writes or IP packets. HTTP/2 and HTTP/3 can add their own multiplexed framing. Offload inside the host adds another distinction between captured units and wire units.

An observer therefore cannot assume that one packet equals one click, one word, or one video frame. Yet aggregate patterns can still be informative. A request followed by a large burst in the opposite direction suggests a different workload from a steady bidirectional call. The uncertainty in that inference is part of the evidence, rather than a reason to treat encrypted traffic as featureless.

9 What the path reveals

Privacy needs an observer

Consider a person reading a medical article over an authenticated HTTPS connection. Assume the browser and server are uncompromised, certificate validation succeeds, and the local access network does not terminate TLS. The connection protects the HTTP request and response from ordinary passive reading between those endpoints. It does not prevent the server from seeing the request it must answer.

A passive observer records communication without changing it. An active observer can inject, alter, delay, or block traffic, within the capabilities granted by its position. A threat model states which observers and capabilities matter, what information should remain protected, and which components must be trusted. Without those assumptions, a statement such as "the traffic is private" cannot be evaluated.

Confidentiality limits disclosure of protected information. Integrity concerns unauthorized alteration and its detection. Authentication provides evidence about a peer or message origin under a specified identity model. Anonymity concerns hiding a participant's identity within a relevant set of possibilities. Unlinkability concerns an observer's ability to associate actions or identifiers with one another. Privacy is the broader interest in controlling or limiting exposure of information about people and their activities. These properties can overlap, but they do not imply one another.

A server can authenticate a logged-in user over a confidential channel. The channel may be secure while the action is explicitly linked to that user's account. Conversely, a service might accept an unnamed request whose unencrypted contents are visible to several networks. Evaluating the properties separately prevents a single reassuring indicator from standing in for the entire analysis.

Each location gets a different view

An observer on the laptop's local link can ordinarily see link-level information relevant to that link and the IP traffic it carries, subject to link encryption and access. A router farther away does not normally receive the laptop's original Ethernet source address, because link framing changes across routed hops. The server can see the address that ultimately connects to it, which may be a translator, proxy, or privacy-network exit rather than the laptop's local address.

An Internet service provider can associate access traffic with the subscriber or connection it serves. For an ordinary encrypted web connection, it can generally observe destination IP addresses and traffic characteristics. Its view of names depends on DNS behavior, handshake protection, and deployment. A shared hosting address may serve many names, so the destination IP is not always an exact identification of the requested page or even the requested site.

The destination service sees the application data delivered to it. It may also receive cookies, authorization tokens, account details, and browser characteristics. A cookie is application state that a browser returns according to web rules; it can link requests even when the connecting network address changes. A network privacy mechanism cannot erase identifiers the application deliberately supplies to the service.

An illustrative comparison makes the boundaries clearer:

Observer	Ordinary authenticated HTTPS connection	Information still outside that view
Local network with suitable capture access	Local link identifiers, IP endpoints, timing, volume, and exposed handshakes or DNS	Encrypted HTTP body and protected headers
Recursive DNS resolver	Queries it receives and client relationship visible to it	Requests resolved elsewhere or answers reused entirely from another cache
Internet provider on the access path	Access identity, remote network endpoints, traffic patterns, and any exposed name information	Properly protected application contents
Destination application	Delivered requests, accounts, cookies, and connecting address	The client's internal forwarding history unless separately exposed

Table 9.1. A bounded example, not a universal capability chart. Endpoint compromise, TLS interception, a proxy, or a different resolution path changes the rows. Sources: [R16], [R21], [R24], [R25], [R47].

Encryption leaves structure

Metadata is information about communication, such as endpoints, times, sizes, and direction, rather than the protected content itself. The division depends on the layer. An encrypted inner IP header can be payload to an outer tunnel, while the outer header remains necessary for routing between tunnel endpoints.

Traffic analysis uses observable communication features to infer activity or relationships. Packet timing, burst sizes, duration, and direction can narrow plausible explanations. A fingerprint is a recognizable collection of characteristics, such as protocol options, packet behavior, or application response patterns. Fingerprinting produces an inference whose reliability depends on the population, observation conditions, and method.

Correlation compares observations at different places or times. If a burst entering a relay system resembles a burst leaving it after a plausible delay, an observer may infer a relationship. Encryption can change the bytes without removing their timing relationship. Aggregation, congestion, padding, and competing users complicate the inference, but they do not justify a claim that correlation has become impossible. Low-latency anonymity systems explicitly confront this limitation. [R37]

The number of alternatives matters. A network address shared by a household offers a different set of possibilities from an overlay used by many unrelated people. An unusual application pattern can reduce that set. Anonymity therefore concerns what the observer can distinguish, rather than simply whether one field contains a person's legal name.

Handshakes have their own exposure

TLS protects different handshake fields at different stages. In ordinary TLS 1.3 without Encrypted Client Hello, the initial ClientHello exposes information including the server name indication when present. The later certificate exchange is encrypted after the relevant handshake keys are established. It is inaccurate to claim that all certificates remain visible in the same way across TLS versions and deployments. [R21, R48]

Encrypted Client Hello, ECH, protects an inner ClientHello using a server public key and associated configuration. Successful use depends on client, server, and configuration support. It does not conceal the destination IP address or make traffic timing disappear. A fallback or failed deployment must be analyzed according to the actual connection rather than the feature's name. [R47]

QUIC encrypts much of its transport information, but observable packet structure remains. Its Initial protection is not intended to hide Initial content

from an observer who can derive the publicly available initial keys. Later protection uses keys from the authenticated handshake. "Encrypted packet" therefore needs a statement about the key and the attacker, not merely the presence of ciphertext. [R18]

Decide what would count as exposure

For the medical-article example, the reader might want to hide the page contents from the Wi-Fi operator, hide the site from the access provider, avoid identifying the account to the server, or prevent two browsing sessions being linked. These are different goals. One mechanism can help with several, but the evidence must follow each goal separately.

A useful threat model can be written as a sentence: under trusted endpoint software, a local network observer should not learn application contents; a particular intermediary is trusted with connection endpoints; a destination may still recognize the account. Such a sentence exposes assumptions that a general promise conceals.

The next chapter considers what happens when an observer can do more than watch. The same boundaries will determine which manipulations are possible and which defenses can constrain them.

10 Interception and manipulation

Capture requires a position

Packet capture records traffic available at an observation point. On the host, software with suitable permissions can inspect packets as the operating system exposes them. On a shared medium, traffic visibility depends on the technology and security. On a switched Ethernet network, an unrelated port does not automatically receive every unicast frame. An attacker may need a mirrored port, a compromised device, a forwarding position, or another way to obtain the relevant traffic.

Sniffing is passive capture and inspection. If application data is unencrypted, the observer can potentially read credentials, messages, and session tokens carried there. If the data is properly encrypted, the capture still supplies metadata and may support later analysis. The ability to record a packet does not by itself establish the ability to decrypt its content.

Capture inside a host also has limitations as evidence. Offload can change what is represented at that point. A packet recorded before a later firewall decision may never reach the physical network. An independent receiver beyond the boundary is often needed to establish that prohibited traffic actually escaped.

Becoming an intermediary

A man-in-the-middle attack places an adversary between participants in a way that lets it observe or manipulate their exchange. The position may come from a malicious access point, compromised router, altered name resolution, routing changes, or local network manipulation. The attack's success depends on what the endpoints validate after traffic reaches the adversary.

ARP poisoning exploits the lack of cryptographic authentication in ordinary ARP. Under suitable local-network conditions, an attacker sends misleading mappings so a victim associates the gateway's IP address with the attacker's MAC address. If traffic is then forwarded onward, communication can continue through the attacker. If it is discarded, the immediate consequence is disruption. A description must distinguish those outcomes rather than treating every misleading ARP message as a demonstrated content theft. [R05]

An authenticating encrypted protocol constrains the intermediary. Redirecting a browser to a different machine does not give that machine the private key for the intended site's valid certificate. If certificate validation is correctly enforced, an ordinary impostor cannot simply complete the same authenticated session. Users ignoring errors, a compromised trust store, stolen credentials, or a managed TLS-interception system create different assumptions.

Link protections and network segmentation can reduce opportunities for local manipulation. They do not replace endpoint authentication over the complete intended path. The first defense controls who can participate locally; the second controls which remote identity the application accepts.

Poisoning a name answer

DNS manipulation can occur at several points. A malicious configured resolver can return a false answer directly. An attacker may alter an unprotected request or response on the path. Cache poisoning attempts to make a resolver store data it should not accept, affecting later clients as well as the immediate exchange.

Classical response-forgery attacks depend on matching the resolver's expected transaction and source conditions closely enough for a forged reply to be accepted. Randomization and stricter response matching make that task harder, while implementation flaws can change the conditions. The security question is the acceptance rule, not merely whether the packet contains a plausible address.

DNSSEC, the DNS Security Extensions, adds authentication for DNS data under a chain of trust. Encrypted DNS protects a transport relationship. They address different attacks. A validated signed answer can still travel visibly, and an encrypted connection can still deliver an answer supplied by a resolver the client should not have trusted. Chapter 12 develops the distinction. [R23, R24, R25]

Routing can change the observer

BGP distributes reachability among autonomous systems, but the original routing model does not cryptographically prove every advertised path. A false origin announcement or a route leak can direct traffic toward an unexpected network or make a destination unreachable. A route leak means reachability is propagated

beyond its intended policy scope; it need not have been created by a malicious operator. [R26]

Resource Public Key Infrastructure, RPKI, supports statements about which autonomous system may originate a prefix. Route Origin Validation uses such information to classify origins. It can constrain particular false-origin claims, but it does not by itself validate the complete AS path or prove that every selected route follows the intended commercial policy. [R27]

Routing manipulation may expose metadata or create a forwarding position without defeating correctly authenticated end-to-end encryption. It can also support denial of service, which makes the service unavailable. Security includes preserving useful service under a stated model, but confidentiality and availability can fail independently.

This book uses these as established attack classes. It does not assign them to a named incident or organization without an incident source. Describing a mechanism that could occur is different from documenting where it did occur.

Sessions extend beyond connections

An application session can persist across several TCP connections. A web service may use a bearer token, a value whose possession is sufficient to authorize an action under its rules. If an attacker obtains that token from unencrypted traffic, endpoint storage, a malicious extension, or another exposure, it may impersonate the session without taking over the original TCP connection.

TCP sequence manipulation is a different form of interference. It concerns acceptance of packets into transport state. A successful injection into an encrypted transport can still fail the cryptographic integrity checks on the carried records. Calling both problems "session hijacking" without stating the layer makes the remedy unclear.

The distinction matters for privacy tools. Changing a network address does not invalidate a stolen application token. Encrypting traffic does not protect a token after a compromised endpoint reads it. A tunnel that terminates before an unencrypted service leaves an exposed segment beyond its endpoint.

Misconfiguration supplies quieter failures

A proxy-aware browser can still launch another program that connects directly. A VPN can cover IPv4 while a usable IPv6 path remains outside it. A firewall can permit an exempt helper that accepts overly broad requests from unprivileged users. A resolver can be correct during stable operation and revert during a network transition.

These failures often arise from incomplete scope rather than broken cryptography. They need a path inventory, lifecycle analysis, and observation of the relevant boundary. A connection attempt failing for an unrelated reason cannot establish that a restriction caused the failure. A test should first establish that the observer and destination would detect an allowed attempt.

A hypothetical vulnerability is a reasoned possibility requiring validation. A demonstrated exploit supplies evidence that the conditions and effect can be produced. An observed incident adds a real occurrence with its own evidentiary record. Keeping those categories distinct helps the reader judge claims without either dismissing plausible risks or converting every possible failure into an asserted event.

11 What a protected channel establishes

Keys give encryption its boundary

Encryption transforms readable data, plaintext, into ciphertext using a cryptographic algorithm and key. A party with the appropriate secret can recover the plaintext. The useful security claim concerns an attacker who lacks that secret and whose computational capabilities fit the design's assumptions. Encrypting data does not make the sending or receiving application unable to read it.

Symmetric cryptography uses shared secret key material. It is suitable for protecting large amounts of traffic once peers have established the necessary secrets. Asymmetric cryptography uses related public and private keys for operations such as signatures or key establishment. A public key can be distributed without granting the corresponding private capability. The exact operations depend on the algorithm; one should not describe every asymmetric mechanism as "encrypt with one key, decrypt with the other."

A digital signature lets a verifier check that a message was authorized by the corresponding private signing key and has not been changed in a way that preserves a valid signature. It does not establish that the message is true, that the signer is honest, or that the public key belongs to the intended person. Those are separate questions of identity and trust.

A cryptographic hash maps data to a fixed-length value with specified security properties. Hashing alone does not authenticate a message: anyone able to replace both the message and its unprotected hash can construct a matching pair. A message authentication code uses a secret to authenticate information among parties who possess that secret. These distinctions explain why a simple packet checksum cannot substitute for cryptographic integrity.

Protecting content and its context

Modern secure-channel designs commonly use authenticated encryption with associated data, AEAD. The operation protects plaintext confidentiality and produces an authentication tag. Associated data is authenticated along with the ciphertext while remaining unencrypted. This lets a protocol bind visible context to protected content. [R22]

Such algorithms typically also use a nonce, a value that must obey the algorithm's uniqueness or other requirements for a given key. A nonce is not necessarily secret. Reusing it incorrectly can destroy properties the design relies on. Good protocol implementations therefore manage counters, key updates, and state rather than asking application authors to invent random cryptographic arrangements for each message.

Authentication tags make unauthorized changes detectable under the assumptions of the construction. The receiver must reject invalid data before treating it as authentic application input. This does not stop an adversary discarding the whole message. Cryptography can detect or prevent acceptance of altered content while leaving denial of service possible.

Randomness also matters. Generating weak private keys or predictable secret values can defeat a sound algorithm. Secure systems obtain randomness from facilities designed for that purpose and handle failure according to their requirements. The algorithm's name is only one part of the implementation's security.

Establishing a shared secret across an untrusted path

Key exchange allows peers to derive shared key material while communicating over a path others can observe. In an ephemeral Diffie-Hellman style exchange, each participant generates a private value and a corresponding public contribution. Combining one's private value with the peer's public contribution yields material from which both can derive matching secrets.

An observer seeing only the public contributions should not be able to derive those secrets under the algorithm's assumptions. But an active intermediary could establish one exchange with the client and another with the server if neither participant authenticates the exchange. Authentication binds the key agreement to the intended identity and conversation.

Ephemeral key agreement can provide forward secrecy: later compromise of a long-term authentication key need not reveal past session secrets, assuming the relevant ephemeral secrets were erased and the negotiated mode provides that property. It is not a promise that a captured endpoint cannot disclose a live session key, or that every possible resumption mode has identical properties.

Key derivation turns shared secret material and protocol context into keys for particular purposes. Separating keys for different directions and stages reduces inappropriate reuse. Binding the handshake transcript, the accumulated handshake messages, helps ensure the participants authenticate the same negotiated conversation.

Certificates connect keys to names

For an ordinary public HTTPS service, the server presents a certificate chain intended to bind a public key to the requested service identity. A certificate authority signs certificates under the trust system used by the client. The browser or other client checks a path to an accepted trust anchor, the service-name match, validity conditions, and the other rules applicable to its implementation.

A certificate is not a general endorsement of a website's conduct. It supplies a particular identity binding. A malicious site can possess a valid certificate for its own name. A browser must compare the identity requested with the certificate's permitted identities rather than accept any correctly signed certificate.

Time and trust-store configuration can affect validation. A badly wrong clock can make a valid certificate appear outside its permitted period. A locally installed trust anchor can authorize interception that a different client would reject. Revocation behavior varies by client and context; a diagram of a certificate chain does not establish that every client makes the same live revocation checks.

If a user dismisses an identity error, the application may proceed under weaker assumptions than the original design. If the application does not validate at all, encryption can protect a conversation with the attacker while failing to authenticate the intended service. A secure channel needs both the cryptographic operations and the correct acceptance policy.

A TLS 1.3 conversation

TLS evolved from SSL, Secure Sockets Layer. SSL is historical terminology for obsolete protocols, rather than the current name of the same protocol version. A certificate casually called an "SSL certificate" can be used in a modern TLS deployment; that label does not establish that SSL is active. [R51]

In a typical certificate-authenticated TLS 1.3 handshake, the client sends a ClientHello with supported parameters and key-exchange information. The server selects parameters and returns its corresponding contribution. Both derive handshake secrets. Later server authentication messages establish the server's identity and bind the exchange, and Finished messages confirm possession of the appropriate handshake secrets and transcript agreement. Application traffic uses separately derived protection keys. [R21]

This is an explanatory sequence, not a packet count. Messages can share transport packets, optional branches can add exchanges, and session resumption changes the details. The current TLS 1.3 specification consulted for this manuscript is RFC 9846, which obsoletes RFC 8446. The earlier RFC remains important historical documentation; a current reference list should identify that relationship. [R21]

Client authentication is optional in many ordinary web deployments. The server authenticates to the client through the certificate mechanism, while a password, token, or other application method identifies the user later. Mutual TLS adds a client-side certificate relationship. It does not automatically make the human using the client anonymous to the server.

Resumption uses suitable prior state to reduce repeated setup. Early data can be replayed under conditions the application must handle. A replay repeats a previously valid message without necessarily forging its cryptographic authentication. An operation such as changing an account balance needs application safeguards if it could otherwise be executed twice. Reduced handshake waiting must preserve the operation's meaning.

Where TLS ends

A reverse proxy may terminate a TLS connection on behalf of a service and open another connection toward an internal server. That creates two protected relationships, possibly with different authentication and visibility. The proxy can read plaintext between them because it is an endpoint of each. Describing the arrangement as "encrypted end to end" would require saying which ends are meant.

An encrypted tunnel can carry a TLS connection without terminating that inner TLS. The outer endpoint sees tunnel-carried packets and metadata; whether it can read the application content depends on the inner protection. Conversely, TLS between an application and a proxy does not automatically protect an unencrypted segment after that proxy.

Cryptographic work consumes CPU time and can add bytes, buffering, and setup exchanges. Its relative cost depends on hardware, algorithms, message sizes, connection reuse, and the rest of the path. On a long network route, waiting may dominate; on a constrained device or high-rate server, computation may dominate. Removing authentication because a connection is slow confuses the symptom with the cause.

TLS supplies a carefully bounded answer to reading and altering application communication. Names, routing, local policy, and application identity still need their own mechanisms and evidence.

12 Protecting names without hiding their limits

Authentic data and private transport

A DNS answer can be false even if it arrives over a reliable connection. It can also be authentic while visible to everyone who can observe its unencrypted path. Protecting name resolution therefore involves at least two questions: whether the data should be trusted and who can learn the question and answer.

DNSSEC signs DNS data so a validating resolver can check it under a chain of trust. The signature concerns record sets and related authenticated denial mechanisms, rather than encrypting the query. A validating chain can begin at a configured trust anchor and follow the authenticated delegation relationship toward the zone. Missing or broken validation must be distinguished from a legitimately unsigned delegation. [R23]

Successful validation does not guarantee that the operator of the zone is benevolent, that its host is uncompromised, or that the application requested the right name. It establishes specified authenticity and integrity properties for the DNS data. The application still needs to authenticate the service under the identity it intended to reach.

DNS over TLS, DoT, protects DNS exchanges using a TLS transport relationship. DNS over HTTPS, DoH, carries DNS exchanges through HTTPS. Proper authentication of the resolver matters in either case. Encrypting a connection to an impostor resolver would conceal the query from some intermediaries while delivering it to the impostor. [R24, R25]

The recursive resolver still receives the query. Its own onward resolution can involve other observers, with behavior depending on resolver implementation and deployment. Query minimization can reduce the name information sent to parts of the hierarchy, but it does not turn a recursive resolver into a party that never learns the client's question.

Moving a trust relationship

Suppose a device changes from its network-provided resolver to an independently selected encrypted resolver. The local network loses direct access to query contents on that protected transport, while the selected resolver gains the client's questions and whatever client association the connection reveals. This can be useful, but the trust moved rather than disappeared.

The resolver's answer also influences the service path. Different answers can select different server locations or reflect local network arrangements. An encrypted resolver outside an organization's network may not know internal names that the organization's resolver serves. A privacy feature can therefore change compatibility and latency as well as visibility.

An application using DoH independently can bypass an operating-system resolver setting. Whether that violates a policy depends on what the policy promised. If the promise is "all conventional DNS uses this local chokepoint," the rule has a narrower scope than "every name query uses this chosen resolver." The second requires accounting for application-level transports too.

Confinement belongs below preferences

A DNS chokepoint is a deliberately constrained place through which specified lookups must pass. A host can direct its own resolver there and enforce ordinary port-53 traffic through kernel policy. Interception can catch queries sent to a foreign resolver address, while a default deny can prevent an unhandled path.

The chokepoint needs correct request-response association. If several queries are outstanding, an implementation must ensure that a response belongs to the expected query and source relationship. Reusing sockets or pooling work can improve efficiency but also changes the state that prevents one response being mistaken for another. Optimization must preserve that acceptance property.

Ghostnector's Tor path combines resolver configuration with a kernel-enforced conventional-DNS chokepoint. Its performance campaign describes a fresh-per-query upstream socket as part of response correlation, and deferred DNS caching rather than adding it simply to improve a headline time. Those are engineering choices in this implementation, not universal requirements for every safe resolver. [G02, G07]

Its I2P mode is different. The documented policy denies ordinary clearnet DNS, and applications use the I2P router's local proxy and naming machinery. The project explicitly does not treat I2P as a transparent substitute for its Tor DNS path. That distinction will make more sense after the I2P architecture chapter. [G05]

Bootstrap and transitions

A protected resolver may need network access before it can resolve ordinary application names. It may need an upstream address or a way to discover one. A privacy router can require bootstrap information before its protected path is ready. The policy has to permit exactly the supporting operation it intends, without accidentally exempting unrelated application traffic.

Ghostnector's installed I2P campaign found that the router's bootstrap resolution interacted with the service identity and resolver setup in ways a simplified diagram did not capture. The later report describes fixes to the service environment and the handling of resolver state when switching networks. These are reported installed-product findings. They show why testing only a resolver library or a fake router is insufficient for the full integration claim. [G01, G03]

A network change can replace resolver information, expire a lease, or expose another protocol family. A disconnect can leave a local resolver pointed at a stopped service. Recovery therefore includes restoring the intended resolver state in an appropriate order, not merely deleting a firewall rule.

Ask the experiment the right question

A foreign-resolver test can send a conventional query toward an address outside the intended path and observe whether that address receives it. A local successful answer alone establishes less: the client might have obtained it from a cache or another path. An external observer and a controlled answer make the inference stronger.

A canary is a deliberately chosen test target with an expected result. A DNS canary can reveal that a particular resolution path supplied the expected answer. It does not prove that every possible query and transport follows the same route. Its value comes from a defined property and from tests that would expose a contradiction.

Name protection combines data authentication, transport confidentiality, path confinement, and operational behavior. A resolver preference establishes only part of that arrangement.

13 Enforcing a path

Preferences and prohibitions

An application preference says which path the application should try. An enforced policy says which paths the system permits. A browser can honor a proxy setting while another program ignores it. If privacy depends on every relevant program using the same path, a setting in one program does not establish the required boundary.

A firewall classifies traffic and applies decisions such as accepting, dropping, rejecting, or redirecting it. Dropping silently discards traffic. Rejecting supplies a failure response where supported, which can help an application stop waiting. Neither is a cryptographic operation. Their purpose is to control communication under the policy.

A default-deny policy refuses traffic unless a specific rule permits it. A default-allow policy permits traffic unless a rule refuses it. The distinction matters when an unexpected protocol, address family, or interface appears. Under a carefully scoped default deny, an unrecognized case tends to lose availability. Under an incomplete deny list, it can become an unintended path.

The default is only part of the claim. Broad exceptions can defeat an otherwise restrictive policy. A privacy router needs network access to contact peers; exempting its identity can be necessary. If untrusted software can run under that identity or make the router perform unrestricted operations, the exception has become a path around the boundary.

Stateful filtering

Stateful rules can use connection-tracking information to recognize related traffic. An established-flow exception is useful for ordinary firewalls, but a policy transition must decide what happens to flows created before protection began.

Leaving them accepted may preserve direct connections the new policy intended to prevent.

Similarly, NAT state can outlive the rules that originally created it. Changing a translation rule does not necessarily change every existing flow in the same way as a new one. An implementation must account for the kernel's flow-state behavior when applying or withdrawing a profile.

nftables lets Linux apply rule changes in transactions. A transaction can make a coherent ruleset replacement visible without exposing intermediate individual-rule states. That is stronger than issuing several unrelated edits, but it does not make the entire application lifecycle atomic. Starting a router, changing resolver configuration, and recording intent involve other components and failure boundaries. [R28]

An invariant is a property a design requires to hold across accepted states. For example, an I2P policy might require that no Tor egress exemption be present at the same time. An invariant checker can reject a policy representation before the renderer submits it to the kernel. Integration tests must still establish that the rendered and installed behavior matches the representation.

Privilege should have a narrow job

Least privilege means giving each component only the authority its job requires. Linux capabilities divide some powers traditionally associated with the administrative root account into more specific grants. CAP_NET_ADMIN, for example, authorizes powerful network-administration operations. Removing unrelated capabilities reduces authority, but possession of a network-administration capability remains a significant trust boundary. [R30]

A small privileged helper can accept constrained requests from an unprivileged controller. Instead of accepting arbitrary firewall text, interface names, or executable paths, it can derive those details from validated internal state. The smaller request language makes it easier to reason about what the caller can cause.

Ghostnector's records describe this split between an unprivileged control plane and helpers for network policy and application namespaces. The application helper is separate because namespace work needs authority the ordinary network-policy helper should not acquire merely for convenience. The launcher drops to the invoking user's identity before user commands run. These are implementation-specific boundaries, documented in the decision records and relevant source. [G04, G16]

Dropping a user ID is not a complete privilege drop if supplementary groups, capabilities, or inherited handles still confer authority. The sequence must account for what survives execution of another program. Ghostnector's installed

campaign found capability and service-sandbox interactions that prevented the intended launch; later tests examined the application's effective privilege sets. A unit test of a launch function alone could not establish the installed unit's complete behavior. [G01, G03]

Isolation changes the available network

A Linux network namespace provides a separate view of networking resources, including interfaces, addresses, routing tables, firewall rules, and sockets. It lets two processes inhabit different network environments while sharing the same kernel. A namespace is not a separate physical machine or protection from a compromised kernel. [R29]

A virtual Ethernet pair, veth, can link namespaces. One endpoint appears in each environment, and traffic sent through one is received at the other. A bridge can join virtual links, but its forwarding behavior must be controlled if applications are meant to remain isolated from one another or from external forwarding.

Ghostnector's APP design uses a namespace with a default route terminating on a dead-end dummy interface. The route exists so the kernel can begin the local connection path, but it cannot carry an ordinary packet to an external network. The project's topology experiment found that a literally route-less design failed before the intended local interception could occur. The security property was therefore expressed as absence of a usable external route, rather than absence of every default route. [G04]

If the relevant redirect disappears, the intended result is local failure instead of fallback through the host's ordinary route. That structural restriction differs from relying exclusively on a mutable host-wide rule. It still needs validation of namespace shape, interfaces, forwarding settings, and the permitted conduit. A namespace with an accidentally added route is a different environment from the one the architecture assumes.

Failure states should say what is known

Fail-open behavior restores or permits ordinary traffic when a protection mechanism fails. Fail-closed behavior continues to deny the traffic that should require protection. Neither phrase establishes what the software actually does at every transition. The relevant questions include what persists after a process exits, when a restrictive baseline appears, and how a person recovers.

Ghostnector separates applied policy from verification. A policy can be installed while current verification is unavailable. Its state model distinguishes that degraded evidence from a contradiction that triggers a blocked state. Scope also matters: blocking APP traffic does not establish that unrelated host applications are blocked. [G02, G16]

State concept	What it establishes	What it does not establish
Off	Protection is not being claimed	A safe route for sensitive work
Applied with incomplete or stale evidence	Policy exists but specified checks are not currently sufficient	Fresh successful verification
Protected under configured checks	Applied policy and qualifying recent evidence under the documented rules	Universal anonymity or a test of every possible path
Blocked	The relevant restrictive failure policy is intended to deny its scope	Availability of the protected service

Table 13.1. A conceptual reading of the project's state distinctions. Actual state reasons and scope remain part of the claim. Sources: [G02], [G16].

A periodic verifier cannot undo traffic that escaped before it detected a privileged rule deletion. The claims document explicitly bounds detection by an interval and timeout and records a fault-injection window in which traffic crossed before re-denial. The host and its administrative components remain trusted. It would be inaccurate to turn "tampering is noticed within a bound" into "tampering can never permit a packet." [G02]

Enforcement has a performance budget

Filtering consumes work, but the cost depends on rule shape, traffic rate, connection state, and the kernel path. A design that launches several administration programs during application startup can incur a noticeable startup cost while adding little work to each later data byte. Those are different optimization targets.

Isolation can add interfaces, forwarding steps, policy checks, and userspace conduits. It can also make a security property less dependent on a single mutable rule. Removing the isolation to improve a startup benchmark would change the property being measured. A valid optimization finds a cheaper way to establish and preserve the same environment.

The permitted intermediary remains a separate trust boundary. Its work and its view of traffic depend on the forwarding protocol it offers.

14 Proxies tunnels and VPNs

Forwarding through another endpoint

A forward proxy accepts a client's request to communicate elsewhere and makes the onward connection on the client's behalf. The destination ordinarily sees the proxy's connecting address. The proxy sees the client's relationship to it and receives enough information to perform the forwarding. Whether it can read application contents depends on where encryption terminates.

An application can explicitly speak a proxy protocol. Alternatively, the operating system can intercept traffic and deliver it to a proxy that recovers the original destination. The latter is often called transparent proxying from the application's perspective. Transparency concerns configuration and interception; it does not imply invisible network behavior or automatic security.

A reverse proxy accepts connections on behalf of one or more services. Clients treat it as the service entry point, while it forwards to internal servers. A reverse proxy may handle load distribution, caching, or TLS termination. Its position differs from a forward proxy chosen by a client to reach unrelated destinations.

In either arrangement, the intermediary changes the communication graph. The original application connection may terminate at the proxy, which then establishes another connection. A measurement of the first connection's local acceptance can omit the second connection's setup. The application can experience that deferred work as waiting for the first response byte.

What SOCKS5 asks for

SOCKS5 is a protocol through which a client requests network forwarding from a proxy. The peers negotiate a method, then the client sends a command and destination information. A CONNECT request asks the server to establish a TCP

connection. The destination can be represented as an IP address or domain name, allowing the proxy to resolve the name when that form is used. [R31]

The protocol also defines other operations, including UDP association, but a particular proxy may support only a subset. The presence of UDP in the SOCKS specification does not establish that Tor carries arbitrary UDP application traffic. Support must be checked at the implementation and architecture level.

SOCKS5 is not inherently an encryption protocol. Its forwarding request and carried connection need protection from another layer if their path requires confidentiality. An application using TLS through SOCKS can retain TLS protection to the remote service, assuming the proxy forwards the stream without terminating that inner security relationship.

Proxy authentication can identify or group clients without encrypting the entire path. Tor can also use SOCKS authentication values as isolation information. In that use, their purpose includes telling Tor which streams should be treated separately under its configured isolation rules. Removing or reusing them to gain connection reuse can therefore change a privacy property even when no user password is involved.

The location of name resolution matters again. A locally resolved IP in CONNECT exposes the local lookup to its resolver. A domain-name CONNECT can shift the lookup to the proxy. A benchmark comparing the two has changed more than the forwarding interface.

Tunnels carry another network relationship

A tunnel encapsulates traffic inside another protocol relationship between tunnel endpoints. The outer headers route traffic to the tunnel peer; the inner packet or message describes the communication being carried. Tunneling does not inherently imply encryption. Encryption depends on the tunnel protocol and configuration.

A virtual private network, VPN, establishes a controlled network relationship across another network, commonly using authenticated encryption. A client may obtain a virtual interface that accepts IP packets. The VPN software or kernel then protects and encapsulates them for a gateway. At the far end, the gateway removes the outer protection and forwards the inner traffic according to policy. IPsec is one family of mechanisms that can supply security at the IP layer. [R32]

The access provider then sees communication with the VPN endpoint, its timing, and its volume. The VPN endpoint can associate the client's tunnel with onward destinations and timing. Application TLS can still conceal application contents

from that endpoint, while an unencrypted inner application exchange may become readable after tunnel termination.

A VPN can therefore reduce what the local network learns while concentrating trust in its operator. An operator's logging and service claims are separate from the protocol's cryptographic properties. This book does not infer those operational practices from the existence of an encrypted tunnel.

The route is part of the VPN

A full-tunnel configuration aims to carry the relevant ordinary traffic through the VPN. A split-tunnel configuration carries only selected destinations or applications. Both can be intentional. The error is presenting one scope while implementing another.

Routes, DNS, IPv6, local-network exceptions, and privileged bypasses all contribute to the effective scope. A tunnel can be up while a particular route still uses another interface. A preferred tunnel route can disappear and reveal an ordinary default route. A kill-switch policy attempts to prevent that fallback, but its actual behavior requires examination during startup, teardown, crashes, and network changes.

The gateway's endpoint must remain reachable outside the tunnel it is establishing. This creates a necessary exception that should be narrow and explicit. A route to the VPN server is not permission for every application to communicate directly with any destination.

If an application opens its own encrypted DNS connection over an allowed route, the query's privacy follows that route and resolver relationship. A VPN changing the system resolver does not establish that every application adopted it. The same distinction between preference and enforcement applies.

Encapsulation consumes capacity

Tunnel headers and authentication data add bytes. If the outer path supports a particular MTU, the inner packet budget may be smaller. Fragmentation, retransmission, or failed path-size discovery can then dominate the experience. The overhead ratio also depends on payload size: a fixed additional header is a larger fraction of a small packet than of a large one.

Userspace processing can introduce additional buffers, syscalls, and scheduling. Kernel implementations have their own code paths and constraints. Neither placement alone establishes the faster implementation. Hardware support, batching, memory access, and workload shape can matter more than a label.

Carrying a reliable transport inside another reliable stream can create interacting retransmission and ordering behavior. If an outer TCP stream loses bytes, inner traffic sharing it can stall while the outer stream repairs the gap. An inner transport may respond to delay in its own way. The magnitude depends on implementation and conditions; it is not sound to claim every TCP-based tunnel always performs badly or every UDP-based tunnel always performs well.

More intermediaries change assumptions

A multi-hop design carries communication through several intermediaries. This can separate knowledge if the parties are distinct and do not combine their observations. Several servers controlled by one operator do not automatically supply that separation. Nor does a longer chain automatically prevent timing correlation.

Every additional relay can add propagation distance, processing, queueing, setup, and another availability dependency. Yet reducing the chain can remove the very separation the privacy design was intended to create. Deciding whether an intermediary is unnecessary requires the threat model, not merely a latency graph.

VPNs and proxies supply useful mechanisms for changing paths and trust relationships. Tor takes the separation further by constructing circuits in which the client establishes layered cryptographic relationships with relays. That architecture deserves its own explanation before any comparison of speed or protection.

15 Tor builds a circuit

Concealing a relationship

Ordinary encrypted communication can conceal a request while leaving the client's address directly associated with the destination's address. Onion routing addresses that relationship by forwarding traffic through a sequence of relays and applying layered cryptography. The aim is to keep an individual relay's normal protocol view narrower than the complete client-to-destination relationship.

The Tor Project's history places early onion-routing work at the U.S. Naval Research Laboratory in the 1990s. The later design paper by Roger Dingledine, Nick Mathewson, and Paul Syverson describes Tor as a practical low-latency system for TCP-based applications. Its design choices explicitly include usability and efficiency alongside anonymity. The paper is historical evidence for the design and its assumptions, rather than a complete specification of every current algorithm. [R37, R38]

Tor is an overlay network: its relays communicate using underlying Internet connections while the overlay creates additional routing and cryptographic relationships. A client normally does not need to replace Internet routing between all those relays. It constructs a circuit through them, and the relays carry the circuit's traffic over their existing network reachability. [R33]

Learning which relays exist

Before choosing a path, a client needs authenticated information about the relay network. Tor's directory system distributes signed information, including a consensus view produced through its authority process. Cached distribution reduces reliance on every client contacting every authority directly. The client validates the relevant information before using it to select relays.

This architecture contains explicit trust assumptions. Signatures authenticate directory statements under the authority arrangement; they do not prove every listed relay is honest. Relay characteristics, selection constraints, and the risk of adversarial participation still matter. A list of available public keys is a necessary input to the system, not a complete anonymity proof.

For an ordinary anonymous connection to an external Internet service, the familiar path uses an entry guard, a middle relay, and an exit relay. This is the canonical example, not a claim that every Tor circuit has exactly that purpose or length. Directory operations, onion services, and specialized configurations have different circuit roles. [R34, R35]

The exit must allow the requested kind of onward connection under its exit policy. The client selects relays using network information and constraints intended to avoid inappropriate combinations, such as using the same relay twice or closely related relays in one path. These rules evolve; manually fixing a convenient path can change both performance and exposure.

Why entry guards persist

The first relay can observe the client's network connection to it. If a client selected an entirely new first relay repeatedly, an adversary controlling some relays would obtain repeated opportunities to occupy that position. Guard selection limits this kind of exposure by using a constrained set over time under the protocol's rules. [R35]

Persistence has tradeoffs. A bad guard remains significant, and observation of a client's guard relationship can matter. The point is to manage exposure across repeated circuit construction, rather than pretend one random selection eliminates adversarial participation. Choosing an entry merely because a short benchmark found it fast ignores the selection policy's security purpose.

The access network may recognize a connection to a publicly known Tor relay. A guard does not by itself hide Tor use from that network. Bridges and pluggable transports address censorship and recognition through additional distribution and transport mechanisms, but they have their own assumptions and are not a promise of undetectability.

Establishing keys along the path

The client establishes a channel to the first relay and performs a circuit handshake. It then extends the circuit toward the next relay using the existing path to carry the extension request. The client derives separate cryptographic material with each hop. Extending again reaches the intended final relay without requiring that final relay to accept an ordinary direct connection from the client. [R34]

The important distinction is between link protection and circuit protection. A protected channel between neighboring Tor participants secures that link. The client's layered circuit cryptography spans several such relationships, so one relay does not simply receive an ordinary decrypted application stream intended for the next relay.

For traffic toward the exit, the client applies the layers in an order that lets the first relay remove its layer, the second remove its layer, and the exit process the final relevant relay message. The relays also apply integrity and protocol checks according to the design. Return traffic uses the corresponding reverse-direction state. The schematic explanation identifies key relationships without claiming each relay message corresponds to a separate application message or IP packet.

Tor organizes traffic into cells and relay messages. The structures help carry many circuits over relay channels and many streams through circuits. Their framing differs from the application's original TCP segments. A relay does not forward the laptop's original Ethernet frame across the entire overlay.

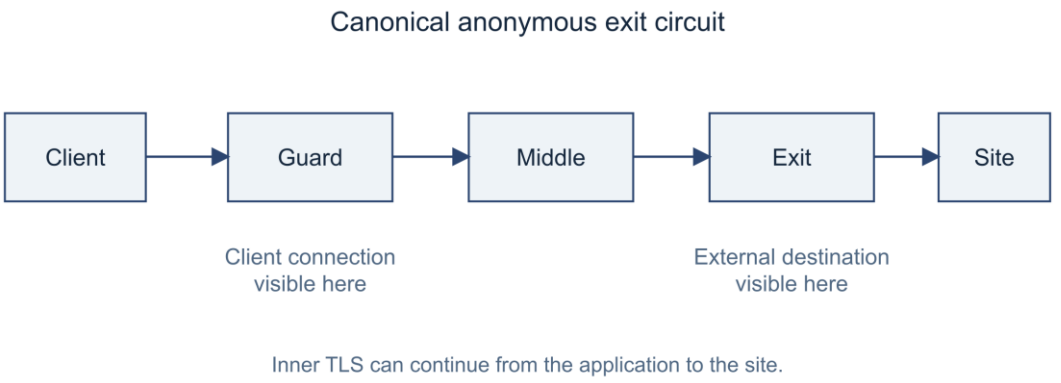


Figure 15.1. The canonical three-relay exit circuit. The guard normally knows the client connection, the exit knows the external destination, and the middle forwards between relays. Collusion and traffic analysis can combine information beyond these individual protocol views. Sources: [R33], [R34], [R35].

Opening an application stream

The client can receive an application connection through SOCKS and ask the exit to open a stream to the requested service. If a hostname is supplied through the supported interface, resolution can occur through the Tor path rather than the application's ordinary local resolver. Transparent arrangements need their own interception and name-resolution integration.

The exit establishes the onward TCP connection. For an ordinary HTTPS request, the application can conduct TLS with the destination through that forwarded stream. The exit then sees the destination relationship and encrypted application bytes, rather than automatically learning the HTTP content. If the application

instead sends plain HTTP, the segment between exit and destination does not acquire HTTPS protection merely because the earlier path used Tor.

No single honest relay following the ordinary protocol view is handed the entire client-origin and external-destination relationship. That statement is narrower than "no one can learn the relationship." The client knows what it requested. The destination can recognize an account. Several cooperating observers may correlate timing across the network. The next chapter examines those boundaries.

Streams may share a circuit subject to the client's isolation rules. Isolation helps prevent traffic that should remain separate from being grouped by the client. Tor Browser applies application-aware behavior beyond simply using a SOCKS port. A system that transparently redirects arbitrary programs cannot assume it has reproduced all of that browser behavior.

Ghostnector supplies distinct source identities and per-group SOCKS credentials on its APP path. Its claims carefully separate the identity information supplied to Tor from an unconditional guarantee about distinct circuits. Later real-Tor tests reported distinct observed exit addresses in particular runs, but a pair of addresses is evidence about those runs, not a theorem that every group always receives a unique exit. [G02, G11]

A circuit consumes time and capacity

Circuit construction performs network exchanges and cryptographic work. Maintaining usable circuits can reduce the delay a later application experiences, while a cold start may include directory acquisition and bootstrap. A measurement starting after Tor is ready answers a different question from a measurement starting with a stopped router.

During use, traffic crosses additional paths and relay queues. The capacity of the bottleneck and the congestion behavior of the combined system constrain throughput. Multiplexing and channel ordering can couple traffic that the application considers separate. Relays contribute CPU and network capacity, and the workload changes over time.

Some cost follows from the privacy architecture's deliberate separation of knowledge. Other cost comes from implementation, scheduling, or resource management. Distinguishing those categories is more useful than treating every millisecond above a direct connection as waste. We will later use that distinction to identify an optimization the surrounding software can make without changing Tor's relay structure.

16 Tor at the boundaries

The exit is a boundary

For an external service, the exit relay leaves the Tor overlay and makes an ordinary network connection. The destination sees the exit's connecting address. It can still see the application request delivered to it and any identifying state included there. The exit can observe destination information and, without inner application encryption, the application content on its onward connection.

TLS remains useful through Tor because it authenticates and protects the application relationship beyond that exit. Tor changes the path and the distribution of endpoint knowledge; TLS protects a different set of information. Combining them can address different observers, provided the application does not defeat its own identity-validation rules.

DNS needs the same boundary analysis. A hostname supplied to the proxy can be resolved through the overlay. A hostname resolved directly by an arbitrary application before it contacts the proxy can be exposed elsewhere. A transparent Tor setup therefore needs deliberate DNS handling and restrictions on unsupported traffic, rather than a routing rule for TCP alone.

The Tor process itself needs ordinary connectivity to relays. An exemption for that process is part of the architecture, while an exemption for every local resolver or helper is not automatically justified. The enforcement layer must distinguish supporting traffic from application traffic it promised to confine.

An onion service removes the ordinary exit

An onion service lets a service participate within Tor without publishing an ordinary connecting IP address to each client. The service creates circuits to introduction points and publishes a descriptor through the onion-service

directory mechanism. A descriptor provides information the client needs to initiate contact under the protocol's authentication rules. [R36]

The client obtains and validates the descriptor, chooses a rendezvous point, and establishes a circuit to it. Through an introduction point, the client supplies the information needed for the service to meet it at that rendezvous. The service then constructs its own circuit to the rendezvous. The joined circuits carry the communication without an ordinary Tor exit connecting to a public service IP address.

The introduction point and rendezvous point have different jobs. The introduction point helps deliver the contact request. The rendezvous point joins the communicating sides after the required exchange. Treating the introduction point as the permanent data exit conceals the location-hiding mechanism.

A version 3 onion address is tied to cryptographic identity rather than being an ordinary DNS alias for the service's IP address. The protocol uses additional descriptor keys and protections, including mechanisms not shown in a simple overview. A reader should understand the identity relationship without assuming that a short drawing lists every current key or directory step.

The familiar anonymous onion-service connection uses circuits from both sides, commonly represented as two three-relay paths meeting at a rendezvous. This adds construction and routing work compared with the canonical exit circuit. Specialized single-onion configurations make different responder-anonymity assumptions and should not be silently substituted into a description of an anonymous service. [R36]

Application security still matters. A service can reveal identifying information in its content, logs, external resource requests, or administration. The protocol's location-hiding properties do not inspect everything the service operator publishes. A client can likewise identify itself by logging in or submitting personal information.

Low latency preserves observable relationships

Tor is designed to support interactive communication. It does not generally hold every message for an arbitrarily long mixing period or produce unlimited cover traffic independent of user activity. Traffic entering and leaving can therefore retain timing and volume relationships.

An adversary with suitable observation near both ends may attempt correlation. It may also create or exploit recognizable traffic patterns. The precise success probability depends on the attack, workload, observation coverage, and defenses; it cannot be read from the number of relays alone. The original Tor design

explicitly excludes protection against an unrestricted global passive observer as a general design claim. [R37]

This is not equivalent to saying the system has no value against any powerful observer. A threat model can include important observers with narrower coverage. The practical question is what information those observers possess and whether the design separates it sufficiently for the intended use. Claiming perfect anonymity discards that analysis; dismissing every bounded protection discards it too.

Adding padding can obscure some size or timing features, but it consumes bandwidth and may not conceal application behavior completely. More delay can frustrate certain comparisons while harming interactive usability. There is no basis for treating unlimited protection as a free setting that implementers forgot to enable.

Browser privacy extends beyond routing

A browser sends application-level signals: cookies, cache state, language preferences, screen and platform characteristics, and the effects of scripts. A site may use combinations of those signals for recognition. Network routing does not remove them automatically.

Tor Browser includes browser-specific protections and isolation choices. The Tor Project's guidance distinguishes using Tor from making an arbitrary application safe to use through Tor. An application can contain direct connections, external helper programs, or protocol fields that disclose information its user expected to conceal. [R49]

This boundary is especially important for transparent tools. Kernel redirection can enforce a network path without rewriting application content. A messaging client that sends its account identity remains identified to that account service. A program that embeds a local address in a payload may disclose it even when the transport route is confined. The path and the application need separate claims.

Faster choices can change exposure

Fixing a preferred exit, shortening a circuit, disabling stream isolation, or reusing state across identities may improve some measurements while changing how observers can connect activities. Such a configuration is a different privacy experiment. It cannot be presented as a free optimization of the original guarantees.

The same warning applies to adding a VPN before or after Tor. The order changes which party knows the client's access identity, which party sees the destination

relationship, and what traffic is available for correlation. It also adds trust and operational dependencies. A chain of product names is not a substitute for drawing the actual path.

Bootstrap delays, circuit reuse, failures, and destination blocking affect availability. Some services restrict Tor exits; a failed request can reflect that policy rather than a local leak. A verification endpoint must remain suitable for its job, and its failure should not be disguised as successful protection. Ghostnector's GUI campaign encountered an unavailable check endpoint and correctly entered a restrictive state even though the harness had expected a protected result. The report classified the mismatch as an expectation problem, while retaining the observed behavior. [G01]

17 I2P connects destinations

Begin with a different object

I2P, the Invisible Internet Project, is an overlay for communication among cryptographic destinations. Its introductory architecture document traces its original description to 2003 and explicitly warns that historical descriptions need checking against later protocol work. That warning matters: an old list of cryptographic algorithms should not be presented as a complete description of a current router. [R39]

A router is the software participant that exchanges I2P network traffic and maintains the resources needed for overlay communication. An I2P Destination is an application-facing cryptographic identity. A router can support destinations without those identities being equivalent to the router's ordinary IP address. Separating router contact information from destination reachability is part of the privacy architecture.

This changes the starting question. For an ordinary Internet service, a client resolves a name to an IP address and opens a transport connection. For an I2P service, the application needs the destination identity and information about how messages can enter its current inbound tunnels. It does not simply obtain the service host's public IP address and connect directly to it.

I2P is therefore not Tor with different relay names. The architecture organizes communication through unidirectional tunnels owned by the sending and receiving sides, with a distributed database supplying the relevant reachability information. [R39, R40, R41]

An outbound tunnel and an inbound tunnel

A tunnel is a temporary unidirectional path through selected I2P routers. In an outbound tunnel, the creator sends messages away from itself toward the tunnel endpoint. In an inbound tunnel, messages move from a gateway toward the creator. The gateway is where a message enters that tunnel; the endpoint is where it leaves.

Suppose Alice's application sends to Bob's destination. Alice's router selects an outbound tunnel. Its endpoint forwards the message to a gateway for one of Bob's inbound tunnels, using the destination reachability information Alice obtained. Bob's inbound tunnel carries the message to Bob's router and the appropriate destination.

For a reply, Bob uses an outbound tunnel and a gateway for one of Alice's inbound tunnels. The reverse message is not required to retrace Alice's outgoing route. A round trip in this ordinary explanatory case therefore uses four tunnel roles: Alice outbound, Bob inbound, Bob outbound, and Alice inbound. This is a statement about direction and ownership, not a fixed count of physical Internet hops. [R40]

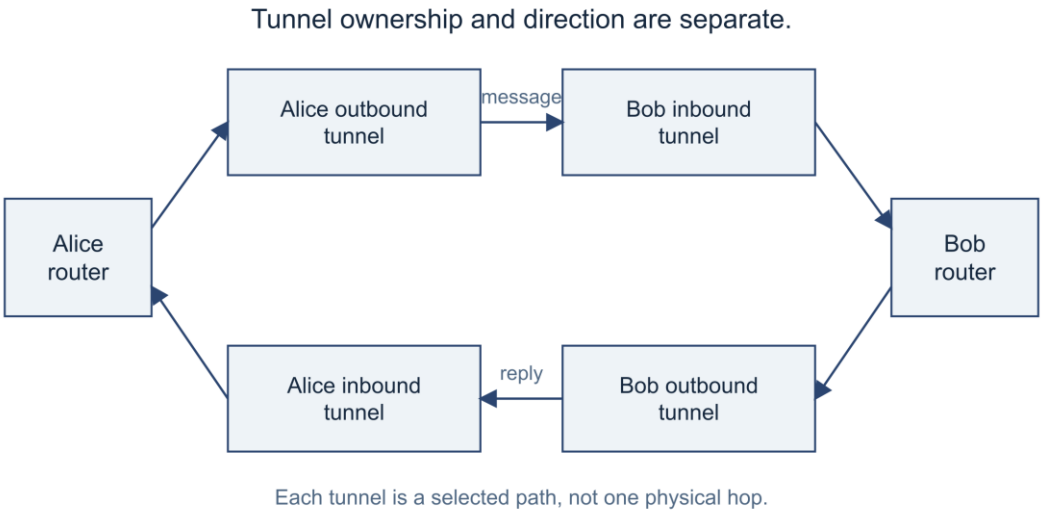


Figure 17.1. A message and reply through the two participants' outbound and inbound tunnels. Each labeled tunnel can contain a configured sequence of participating routers. The drawing intentionally does not assign a universal tunnel length. Source: [R40].

Each creator maintains tunnel pools so communication need not wait for a completely fresh tunnel on every message. Tunnels expire and are rebuilt. Construction, testing, peer selection, and replacement are ongoing work. A warm router with established pools has a different startup state from a newly launched router still learning peers and building usable paths.

Tunnel length is a privacy and performance choice within the implementation's constraints. A zero-hop or short tunnel changes which peers can associate activity with the creator. It must not be compared with a longer anonymous configuration as though only an inefficient delay had been removed.

Construction establishes forwarding state

The creator selects participating routers and arranges tunnel state, including the information and cryptographic material each participant needs. Tunnel-build mechanisms use protected records so participants receive their own instructions without being handed the whole creator-selected path in ordinary operation. The details depend on the tunnel-build specification and supported generation of the protocol.

Once established, the tunnel carries messages using layered transformations and per-hop state. Tunnel protection should be distinguished from both neighbor-to-neighbor transport protection and end-to-end destination message protection. They have different endpoints and expose different information when one participant is malicious.

Large overlay messages can be fragmented into tunnel-carried units and reassembled where the protocol requires. That introduces framing, buffering, and loss consequences independent of the application's original message boundaries. A single application operation can therefore depend on several carried units arriving through several underlying Internet links.

An intermediate participant needs enough information to perform its forwarding role. It does not ordinarily need an application's readable request or the complete relationship between the originating and receiving destinations. As with Tor, that limited protocol view does not defeat every inference from timing, topology, collusion, or hostile participation.

RouterInfo and LeaseSets answer different questions

I2P's network database, netDb, distributes signed information. RouterInfo describes how to contact a router and relevant cryptographic or transport information. A LeaseSet describes reachability for a destination, including current inbound-tunnel information and material needed for protected communication. The structures have lifetimes because reachability changes. [R41]

A subset of routers called floodfill routers participates in storing and distributing this information. Lookup and storage use the network database's specified mechanisms. This is different from Tor's signed directory consensus. It should

not be described as a single global list from which every router receives the same complete view.

Signing an entry establishes who authorized the data under its key; it does not make the storage infrastructure immune to withholding, selective responses, or hostile participation. Redundancy, expiration, lookup rules, and peer knowledge all contribute to reliability and resistance. The database is part of the threat model because an adversary that influences discovery can influence who becomes reachable or selected.

The creator's tunnel choices also depend on its knowledge and assessment of peers. Local performance profiling can help maintain useful paths. Favoring apparently excellent peers, however, interacts with adversarial selection risks. The fastest observed peer is not automatically the safest choice when an adversary can influence the observations.

Garlic is a message mechanism too

Garlic terminology refers to related mechanisms in I2P, including layered protection and the bundling of messages called cloves. A garlic message can carry application-related data and supporting information with delivery instructions. Bundling can convey information the recipient needs for replies or delivery status without requiring an unrelated public lookup every time. [R42]

It is misleading to explain garlic routing as automatically combining many unrelated users' messages into one anonymous batch. A bundle can contain one application clove and supporting information. Bundling alone does not establish high-latency mixing or protection against a global timing observer.

The older overview uses historical ElGamal/AES terminology. The separate ECIES-X25519-AEAD-Ratchet specification describes modern end-to-end message protection with X25519 key agreement, authenticated encryption, and evolving session state. A ratchet advances cryptographic state so successive messages need not reuse the same secret in the same way. This is a particular message-protection layer, not a claim that every I2P layer uses one identical algorithm. [R43]

Keeping the layers separate avoids two errors: treating an old architecture page's entire cipher list as current, and treating a current message specification as if it also defined every transport and tunnel operation. A technical book should give the reader the relationships and direct them to the relevant specification for exact formats.

Underlying transports remain ordinary network traffic

The architecture documentation consulted here describes NTCP2 over TCP and SSU2 over UDP as current I2P transports, replacing older NTCP and SSU. Those transports protect communication between routers and handle their own network requirements. The fact that an underlying router link uses UDP does not mean the application receives only UDP's original service model. [R39]

I2P's streaming library supplies TCP-like reliable streams above the overlay for applications. It has its own sequencing, windows, and congestion behavior. Datagram facilities support other application needs. These are distinct from simply forwarding an arbitrary operating-system TCP socket to an external Internet address. [R44]

A client application can use a local I2P proxy or API. The router translates that application relationship into destination-oriented overlay communication. The local proxy's bind address and permissions remain important: exposing it beyond the intended host can create an unintended service boundary.

18 Names trust and cost in I2P

A readable name needs a binding

An I2P destination identity is not an ordinary DNS name. Human-readable .i2p names can be associated with destinations through naming and address-book mechanisms. Those associations introduce their own trust and distribution questions. Two local address books need not begin with the same entries or have learned an entry from the same source. [R39]

Base32-style destination references provide another way to identify the destination relationship. Their form should not be confused with a DNS record that reveals a public IP address. Applications and routers interpret these names under I2P's rules.

An address helper or imported address-book entry can make a service convenient to reach, but convenience does not authenticate the human organization behind it. A cryptographic destination proves continuity with its key, while a user still needs a trustworthy way to decide that the key is the intended service. The same separation between key possession and real-world meaning appeared in the TLS chapter.

For privacy, name resolution through an ordinary clearnet resolver would be the wrong operation for an internal destination. The application should use the I2P naming and proxy facilities appropriate to that destination. A system-wide DNS setting cannot, by itself, make an application understand the overlay's application interface.

Internal services and external gateways

I2P focuses on communication within its overlay. A service hosted at an I2P destination can be reached through its inbound tunnels without revealing an

ordinary destination IP address to each client. An outproxy, when available and configured, can provide access to an external service. That outproxy is another trust and observation boundary, rather than a universal role every I2P path contains.

Tor's canonical exit circuit is consequently not a direct counterpart to every I2P connection. Comparing Tor access to an external website with I2P access to an internal destination changes the destination and the boundary conditions. A fairer internal-service comparison would identify the actual Tor onion-service path and the actual I2P destination path.

Even then, the services should be equivalent workloads with comparable server placement, payloads, application behavior, and startup state. The overlays select different peers and routes, so repeated measurements and uncertainty remain necessary. A claim that one architecture is universally faster cannot be supported by a convenient request to two unrelated sites.

Participants contribute resources

An I2P router can participate in carrying other tunnel traffic according to its configuration and operating conditions. This contributes capacity to the network and creates resource demands on the host. Router startup, peer discovery, tunnel construction, cryptographic work, and participation traffic should be distinguished from the incremental cost of one application request.

For sustained operation, memory and CPU measurements need a stated window and workload. A process's resident memory can vary as it learns peers or holds buffers. A traffic counter can include participation traffic that is not the user's own payload. Measuring total router bytes as though they were application throughput would confuse different quantities.

NAT and firewall conditions influence router reachability and the available transports. A configuration that works on a permissive network may behave differently behind a restrictive gateway. The architecture includes mechanisms for these conditions, but a successful startup on one host does not establish compatibility with every network.

Low latency still has limits

Unidirectional tunnels distribute roles differently from a bidirectional circuit, and destination identity differs from ordinary IP identity. Neither difference erases timing and volume. An observer with appropriate coverage can attempt correlation; hostile peers can influence discovery, availability, or path observations. The netDb documentation itself discusses threats including Sybil

participation and lookup-related attacks. A Sybil attack uses many apparent identities under one adversary's control. [R41]

The relevant anonymity set is not just the total number of router identities. It depends on which participants plausibly produced a particular observation and what the observer already knows. A rare service, unusual traffic schedule, or compromised endpoint can narrow the alternatives. No numerical anonymity score follows automatically from the displayed peer count.

Longer tunnels can change exposure and cost. More concurrent tunnels can improve availability while consuming more resources. Aggressive peer selection can improve immediate performance while concentrating traffic. These choices require an analysis of the preserved threat model and cannot be resolved by selecting the smallest latency number.

Garlic bundling also has bounded benefits. Carrying reply information can avoid some later lookup work, but a bundle has framing and cryptographic costs. The benefit depends on what would otherwise have happened, not simply on whether several messages fit into one named structure.

Ghostnector deliberately uses a different integration

Ghostnector's I2P profile permits the router's own network activity while denying ordinary applications direct clearnet egress. Applications reach I2P through loopback proxies. The decision record explicitly rejects mixed Tor and I2P operation, I2P outside system scope, and a LAN exception that would contradict this confinement model. APP plus I2P is refused because the required per-application conduit was not implemented. [G05]

There is no transparent I2P redirect equivalent to the product's Tor TransPort arrangement. I2P mode has no corresponding NAT redirect chain or Tor-style DNS chokepoint. Describing it as "all applications transparently use I2P" would therefore be wrong. The restriction prevents direct external access, while application compatibility still depends on using the local I2P interface.

Verification requires a configured I2P canary fetched through the router's proxy. Without that evidence, the product remains degraded. A fake-router suite tests deterministic policy and state behavior; a separate real-i2pd qualification exercises actual router integration. The supplied decision record reports 29 held, zero contradicted, and zero inconclusive cases for its native real-router run using i2pd 2.61.0. That is a reported campaign result, not a test rerun for this manuscript. [G05]

The same record describes an older distribution-packaged router crashing in that qualification environment. That finding should not become a universal statement

that every installation of that version fails, nor a permanent recommendation about the newest package available today. Its value is showing why an integration test must identify the actual binary and environment.

Compare properties before products

Question	Canonical Tor external-service path	Ordinary I2P internal-destination path
What identifies the application destination	A hostname or address for the external service	A cryptographic Destination and its reachability information
How the overlay organizes the path	A client-built relay circuit with an exit	Sender outbound and recipient inbound tunnels
Where external forwarding occurs	At the exit relay	Only if a separate external gateway is used
How reachability knowledge is distributed	Tor directory mechanisms	netDb RouterInfo and LeaseSet mechanisms
What must still be analyzed	Endpoints, exit boundary, application identity, correlation	Endpoints, naming trust, database and peer influence, correlation

Table 18.1. Deliberately different use cases. This comparison identifies architectural objects and does not rank speed or anonymity. Sources: [R33], [R34], [R39], [R40], [R41].

The useful comparison begins by choosing the same privacy objective and service relationship. Only then can measurements tell us what each implementation spends to provide it. The next chapters turn to an implementation's lifecycle and to the evidence needed to separate useful engineering from a persuasive but unsupported claim.

19 A path must survive its lifecycle

A steady-state diagram is incomplete

A privacy mechanism spends only part of its life in a stable connected state. It starts, applies policy, waits for supporting services, verifies them, changes networks, handles process exits, and eventually stands down. A guarantee stated only for the middle of that sequence leaves the reader unable to judge the moments most likely to expose an unintended path.

Ghostnector's records make this visible because they preserve both proposed architectures and later qualification failures. The implementation plan lists intended milestones and gates. The handoff documents capture what a later development stage inherited. The final release-candidate report and post-optimization notes identify the product baseline actually tested. These documents have different jobs and cannot be flattened into one timeless description. [G01, G03, G14, G15, G16]

The supplied repository's latest commit is 17a03c4, which finalizes release metadata. The later qualification section names 9b0fe5d as the tested product baseline, superseding f82adf9. Subsequent qualification-tool and documentation changes do not mean every test was newly run against an independently changed packet path. The book follows those distinctions rather than calling every recorded result a test of an unspecified "latest version." [G01, G08]

Deny before constructing the useful path

A fail-closed startup can first establish a restrictive baseline, then prepare the router and permitted conduit, then claim usable protection only after the required evidence appears. If construction fails halfway, the system should retain the intended restriction or remove partial objects without creating a direct fallback.

This is an ordering property. Starting a proxy and applying a firewall afterward can leave a window before enforcement. Applying a permit before its corresponding isolation exists can expose an unintended path. Atomic rule replacement helps within the kernel policy, while the controller must coordinate the other operations around it.

The reverse transition also needs ordering. If a disconnect removes enforcement before the reported state withdraws its protection claim, an observer can be told that protection exists when it no longer does. Ghostnector's historical D-19 defect described precisely such a brief mismatch. The documented correction announces the transition before touching the protected state. [G02, G13]

A status display should therefore render authoritative observed state, rather than infer protection from a selected switch or a running process. A process can be alive while its listener is unusable; a listener can answer while the onward route is broken. A verification result can be stale. These are different facts.

The namespace that lost a destination

One of the most instructive later defects was D-50. The APP design originally redirected a connection inside an application namespace toward Tor's transparent listener in the host namespace. The architecture preserved a source identity and prevented a direct route, but real Tor could not recover the original destination as expected. [G01, G03, G11]

Destination NAT had occurred in the application's networking environment. Its connection-tracking state held the original destination tuple there. Tor ran in a different namespace, whose connection-tracking context had not observed that pre-translation tuple. A connection arriving at a listener was therefore insufficient evidence that the transparent contract worked. The final service needed the intended remote destination, not merely an accepted local socket.

The correction placed a small relay inside each application namespace. The local redirect targets that relay. It reads the original destination from the same namespace's connection state and makes an explicit SOCKS CONNECT request toward Tor's SOCKS listener in the core environment. That carries the destination as protocol data instead of expecting another namespace to reconstruct missing translation history.

The relay runs as the application's unprivileged identity with capability sets cleared. It refuses a connection that lacks the required original-destination relationship. Its onward connection preserves the application address and uses per-group SOCKS isolation credentials. APP-mode Tor no longer relies on the old transparent listener arrangement. The source comments and implementation

provide direct evidence of that intended mechanism; the real-Tor outcome counts remain reported qualification results. [G11]

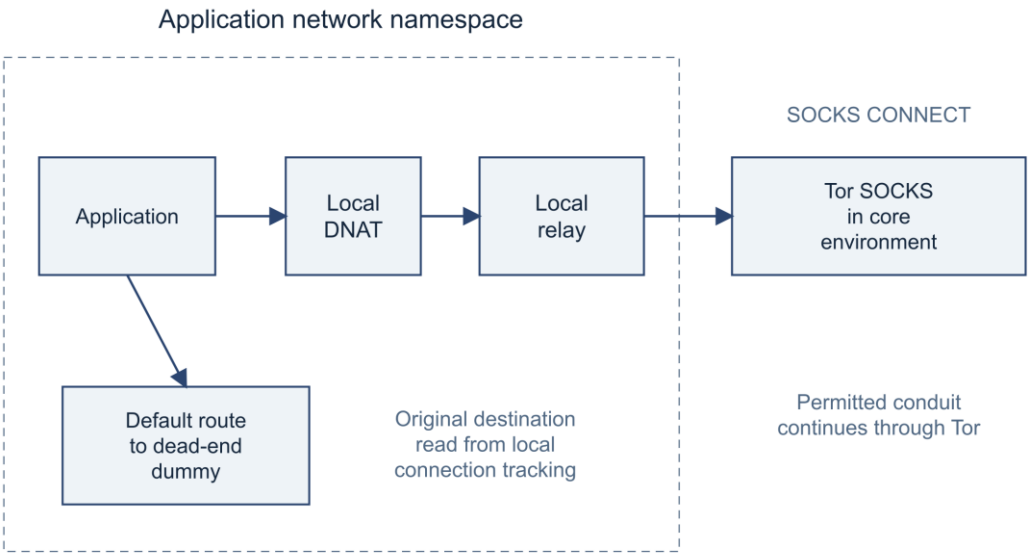


Figure 19.1. Final documented APP path. The relay recovers the original destination where the redirect occurred and supplies it explicitly to Tor through SOCKS. The dummy route provides no ordinary external path. Sources: [G01], [G04], [G11].

The earlier M8 decision text retains some original routing language and adds a D-50 amendment. That chronology is meaningful. The historical diagram explains why the defect occurred; the amendment and later source explain the final mechanism. Silently choosing one sentence from the older body would produce the wrong present-tense description.

A process can keep an environment alive

Deleting a named namespace does not guarantee that every process using it has exited. References held by processes or other objects can keep the networking environment alive. A cleanup design must stop the relevant processes and remove resources in a way consistent with those lifetime rules.

The APP relay's source describes a shutdown pipe held by its helper. Closing the pipe makes the relay see end-of-file and exit. That mechanism matters because the packaged helper does not have unrestricted signaling authority for every target identity. Cleanup follows the actual privilege and reference model rather than assuming that removing a name kills everything associated with it. [G11]

A different lifetime defect appeared during requalification: the command-line session could keep waiting when its command had ended but the caller's standard input remained open. The final product fix returns when the session

ends. This changes process lifetime rather than the protected packet path or the earlier application-execution timestamp. Its position after the launch optimization should not be mistaken for a new relay optimization. [G01, G08]

Restart and boot need persistent intent

Kernel firewall state can survive the death of an ordinary controller process, but a reboot recreates the system. If the user's protection intent should persist, some durable record must guide startup. That record is an instruction to reconstruct or deny, rather than proof that the kernel already contains the required state.

Ghostnector's boot guard applies a restrictive baseline when protected intent exists and the complete path is not yet established. Its installed qualification found D-54 in the guard's ability to reach the helper and obtain current policy state under its packaged identity. The report describes a correction and a rerun asserting the guard's success and completion at or before the network-pre barrier. [G01, G03]

The claims document still lists direct proof that no packet left before denial as an open boot-ordering gap, G6. A successful service-ordering assertion supports an important property, but it is not the same observation as capturing all pre-denial network traffic. The two statements should coexist until evidence closes the latter gap. [G02]

Similarly, later far-side qualification improves on older handoff statements that no external observation existed. That improvement applies to the recorded campaign and its tested windows. It does not retroactively turn every old local observation into an external one, or establish every possible external question.

Recovery is part of usable protection

A restrictive system can intentionally cut off the remote channel used to administer it. A recovery plan must therefore exist outside the path it may deny. Ghostnector's qualification ran in a virtual machine, or VM, a software-created computer with its own operating-system environment. It used guest-control access as an out-of-band recovery channel when the ordinary Secure Shell, or SSH, remote-administration connection was intentionally cut off. The records also document VM reset limitations. [G01, G03]

For the user, recovery should explicitly withdraw protected intent and restore the system state the tool owns. The recovery document's preferred operation is a local stand-down through the controller. A documented boot override and rescue-shell procedure exist for deeper failures. These actions deliberately return to an unprotected state; they are recovery choices, not evidence that protected traffic resumed safely. [G12]

A failure can also be an availability cost without being a leak. A transient network problem can cause a contradiction and leave the system blocked until a person acts. Automatic reopening would improve apparent availability by changing the fail-closed policy. The book treats that as a tradeoff requiring a decision, rather than an obvious repair.

Qualification must try the installed combination

Fake services are valuable for deterministic tests. They let a test control failures and expected answers without depending on a changing public network. But a fake listener can accept a connection that real Tor later resets because the original destination is missing. A test of that fake service does not establish real-Tor integration.

Service-unit restrictions can also affect child processes in ways a direct developer invocation does not. The installed campaign found restrictions involving filesystem access, namespace creation, identity changes, and address families. These defects were exposed by the packaged combination, not by a protocol definition alone. [G01, G03]

The report's final section records a full gate of 21 successful steps and 463 unit tests, with separately identified adversarial and installed runs. Some raw logs named there are absent from the supplied repository. Those counts should be attributed to the report, and the remaining inconclusive IPv6 case in the broader adversarial suite should remain visible. Counting an untestable IPv6 condition as a pass would make the evidence weaker while making the table look better.

These lifecycle properties define the constraints on an optimization. A faster implementation still needs the same ordering, confinement, evidence, and recovery behavior.

ENGINEERING

20 Measuring the cost of privacy

Define the outcome before the stopwatch

"Faster" can mean that a process appears sooner, a first response byte arrives sooner, a large transfer finishes sooner, or less CPU time is consumed. These outcomes can move independently. A tool can reduce application startup while leaving network throughput unchanged. It can improve throughput by increasing buffers while worsening interactive latency.

Latency measures elapsed time between named events. Throughput measures delivered data per unit time under a specified accounting boundary. Goodput counts useful application payload rather than every protocol byte. CPU time measures processor execution consumed by the work. Wall-clock time includes waiting. Resident set size, RSS, describes resident process memory under the operating system's accounting; it is not a complete allocation history or a sum that can always be interpreted without considering sharing. This RSS is a memory metric, distinct from Receive Side Scaling in chapter 2.

The first task in an experiment is to define those boundaries. For application launch, the start might be the controller request, the helper receiving it, the executable beginning, or the application declaring readiness. Those are different measurements. For HTTP, connect completion, first response byte, body completion, and browser rendering are different milestones.

Compare equivalent work

A direct connection answers how much faster the ordinary path is under the observed conditions. It does not isolate the overhead of software wrapped around Tor. An equivalent Tor baseline should preserve the relevant destination, resolution method, transport arrangement, and privacy properties while removing the component whose incremental cost is under investigation.

Ghostnector's early installed report compared product HTTP medians of 0.896 seconds with a standalone Tor baseline of 0.803 seconds. Its later campaign identified architectural and timing differences in that baseline, including hostname resolution through SOCKS and sequential sampling. The 93-millisecond difference therefore should not be presented as a measured intrinsic cost of Ghostnector's code. [G01, G07]

Two Tor instances can also select different circuits and encounter different queues. In one campaign run, the product's route appeared faster overall while the standalone instance resolved DNS faster and delivered the HTTP response more slowly. That pattern points to instance and network variation rather than a general product advantage.

The same-instance experiment instead directed the comparison route into the product's own Tor instance. This controls an important source of variation, though it does not make every stream or network condition identical. Interleaving and randomizing order reduce systematic time effects; they do not create perfect pairs that share all hidden state.

Preserve the data categories

The supplied same-instance log contains 40 latency iterations for each route and three additional throughput iterations for each route. Both categories reuse iteration numbers beginning at one. The log and summary are locally accessible; the separate comma-separated values file, or CSV, named in the log is not included in this source set. For this manuscript, the 129 printed sample rows were reconstructed into a clearly identified derived CSV. [G09]

Inspection of the two supplied analysis scripts revealed a pairing problem. They match route samples by iteration number alone without first restricting the input to latency rows. One script retains the first matching comparison row, while the other retains the last. Because throughput iterations reuse numbers, those choices produce different invalid cross-category pairs for part of the calculation.

The narrative campaign reports a paired median of +18.6 milliseconds with a 95 percent bootstrap interval of -8.2 to +33.1 milliseconds. The raw summary reports +22.9 milliseconds across its mixed 43-row calculation. The manuscript does not silently select one. It identifies the category collision and recomputes only matching latency rows. [G07, G09]

Quantity	Latency-only recomputation
Valid paired latency iterations	40
Product latency median	572.87 ms
Equivalent-route latency median	552.00 ms
Median of paired product-minus-equivalent differences	+21.905 ms
Bootstrap interval for that paired median	-1.40 to +35.31 ms
Two-sided sign-test p value	Approximately 0.154

Table 20.1. New descriptive analysis for the manuscript, reconstructed from the supplied same-instance log. The interval uses the supplied analysis routine's 10,000 bootstrap resamples and fixed seed. It is not a new network experiment. Sources: [G09], [G18].

The difference between the two route medians is 20.87 milliseconds; the median paired difference is 21.905 milliseconds. They need not agree because subtraction and taking a median are different operations. The interval crosses zero, and the sign test does not establish a directional effect at a conventional 0.05 threshold. Neither observation proves zero overhead or a tight universal upper bound.

Adjacent network samples are not necessarily independent. Bootstrap and sign-test calculations inherit assumptions about the observations, and a single VM run cannot represent all relay conditions. The most defensible conclusion is that this run does not clearly resolve the wrapper's total HTTP increment against its variation. The audit itself also demonstrates why a printed confidence interval deserves inspection of how its pairs were formed.

Decompose before attributing

End-to-end elapsed time can be divided into phases such as resolution, local connection acceptance, proxy negotiation, first response byte, and body transfer. The boundaries must remain comparable. A local relay accepting immediately can defer its SOCKS connection work until after the application's connect call, moving cost into a later phase without removing it.

The campaign's focused DNS investigation used loopback capture to separate the chokepoint's local processing from the upstream Tor wait. Its report describes about 1.9 milliseconds into the relay and 0.5 milliseconds out, while the upstream wait was much larger and variable. Those segment figures are attributed to the

campaign analysis, rather than treated as a fresh independent reconstruction of its absent capture. [G07]

A lower local component can coexist with a slower total query because the upstream path changed. Conversely, an apparently faster total does not prove the local component became cheaper. Instrumenting relevant boundaries is how the experiment distinguishes them.

The APP relay comparison reported a +62.6-millisecond paired median and several long outliers against direct SOCKS in that particular run. The report interprets the difference as consistent with isolation and setup behavior, rather than a measured byte-copy bottleneck. "Consistent with" identifies an interpretation; it does not establish a causal proof that only one mechanism produced every outlier. [G07]

Measure the observer too

Monitoring can disturb the system it measures. Ghostnector's status operation obtains kernel policy information and had a measured median around 90.7 milliseconds on the campaign VM. Polling it repeatedly inside a benchmark can add work to the same processes and tools whose cost is being estimated. The revised campaign removed that polling from measurement windows. [G07]

The report also corrected CPU accounting. A process lifetime-average percentage from the process-reporting tool ps is not the same as CPU used during a selected idle window. The later method used process CPU-counter differences over an elapsed interval. It documented the chosen verification interval, which was extended beyond the measurement window. An idle CPU figure measured under that setting should not be presented as the cost of every production verification schedule.

Clock choice matters. A monotonic clock is suitable for measuring elapsed durations without wall-clock adjustments. Comparing events across machines additionally needs synchronization or an offset model. The qualification records preserved an incomplete leakage run when clock correlation could not be established. Later analysis used a contemporaneous offset rather than treating uncertain ordering as a clean pass. [G01, G03]

An observer should demonstrate liveness throughout the relevant window. If a receiver saw no prohibited packets because it had stopped recording, the absence proves nothing. Open validation arrivals, self-probes, and heartbeats can establish that the measurement system was capable of observing traffic. They are controls for the experiment, rather than traffic that should be mislabeled as application leakage.

Distributions need enough observations

The median is the middle observation in an ordered sample, or the average of the two middle values for an even sample. It resists a few extreme values better than a mean. Percentiles describe positions in a distribution. A p90 is not a confidence bound on the median; it describes a different quantity.

With ten launch observations, p10 and p90 provide a limited description of observed spread. They do not characterize rare production failures. A benchmark should keep sample counts and failed attempts, identify warmup and exclusions, and explain any stopping rule. Discarding slow attempts because they "look wrong" can remove the behavior the experiment ought to explain.

For throughput, decimal megabytes and binary mebibytes need explicit units. One MiB is 1,048,576 bytes; one decimal MB is 1,000,000 bytes. A file called 1MB can include framing or application overhead different from either label. Compare the actual payload and measured interval before interpreting a rate.

Reproducibility includes what is missing

An experiment record should identify the implementation commit, installed binary, environment, workload, configuration, observer, and raw data location. It should distinguish a measured result from the explanation proposed for it. If a raw log is missing, a report can still be cited as a report, but the reader should not be told the raw evidence was inspected.

The final installed launch log and several final leakage logs named in the release report are absent here. The book preserves those results with attribution and supplies the accessible source hashes and its own derived latency audit. This allows a later editor with the original VM records to extend the verification without confusing a reconstruction with the original data.

Good measurement narrows the next engineering decision. In Ghostnector's case, it redirected attention from uncertain steady-state forwarding differences toward a repeatable local cost: constructing the protected application environment.

21 Reduce the work without weakening the boundary

Find a cost the implementation controls

An overlay can be slow because of a distant relay or a busy external network. Software around that overlay cannot remove physical propagation or grant the relay more capacity. It can, however, waste local time by repeatedly launching administration programs, waiting longer than necessary for readiness, or copying data it could safely reference.

Ghostnector's campaign reports an APP process-appearance median of 665.6 milliseconds against 50.6 milliseconds for direct launch in the baseline run. The approximate added cost was therefore 615 milliseconds as a difference of those medians. Its reported phase profile placed much of that time between namespace creation and relay appearance, where the helper invoked many networking tools. [G07, G08]

The accessible earlier log named app-launch-profile-20260928T152100Z ends with zero usable product samples, so that file cannot independently substantiate the reported baseline. A later accessible staged-build log, app-launch-profile-20260928T161456Z, does contain ten usable product samples and the 320.9-millisecond median discussed below. This source distinction remains visible rather than treating all named raw records as equally complete. [G10]

The report measured individual tool invocations in the same VM and service environment. The observed cost of repeated process launches made them a plausible target. This was a stronger basis for work than an assumption that all privacy overhead must come from encryption.

Process creation includes establishing execution state, loading the program, applying service restrictions, parsing inputs, performing the requested operation, and collecting completion. A small useful operation can therefore be dominated

by the machinery needed to request it. Eliminating repeated requests can improve startup while leaving the eventual network topology unchanged.

Batch the administration not the guarantees

The first optimization grouped ip operations into a batch per namespace phase and replaced specified interface-presence tool calls with reads from /sys/class/net. The system still had to ask whether the required interface existed; it used a less expensive mechanism to answer that question. The intended namespace shape and later verification remained. [G08, G11]

A batch is not automatically an atomic transaction. The ip program can execute several operations before a later one fails. A safe construction path must account for partial completion and retain the restriction that prevents external traffic. Rollback must remove the objects already created without opening a direct route for the application.

Order also remains significant. The recorded sequence creates the namespace and link, attaches the link, establishes isolation, and only then brings it into the intended active state. Folding bridge isolation into the batch reduced another tool invocation without moving the isolation step after activation. A performance refactor that changed that ordering would require a different safety analysis.

The launch record reports a median of 423.0 milliseconds after the first batching work, compared with 665.6 milliseconds for the qualified baseline. Their 242.6-millisecond difference is a difference between measured build medians. It is not a universal per-launch saving or an independently randomized causal estimate with all external variation removed. The local mechanism and phase measurements support the interpretation that fewer tool invocations removed substantial work. [G08]

Stop waiting after readiness exists

A fixed polling interval can introduce avoidable delay. If a relay becomes ready just after a poll, a 100-millisecond sleep can make the caller wait nearly the full interval before checking again. The average effect depends on the readiness distribution and scheduling; it should be measured rather than inferred as a constant 100-millisecond cost.

The second optimization shortened the readiness polling interval to 10 milliseconds and failed promptly when the relay process had already exited. It retained an overall deadline. This distinguishes three states: not ready yet, dead, and ready. Treating all three as a file or process merely existing would weaken the readiness claim.

The measured build median after the combined first and second changes was 362.4 milliseconds. The report attributes a 60.6-millisecond total difference to that build step while noting noise in another phase. A later refinement folding bridge configuration into the batch produced a 320.9-millisecond median. Those numbers belong to the staged optimization measurements. [G08]

Shorter polling also increases the number of possible checks while waiting. It trades a small amount of additional checking for reduced detection delay. An event-driven readiness signal might remove polling but would introduce its own interface and failure conditions. The appropriate choice depends on the measured cost and the evidence needed to trust the signal.

The final installed result is a separate run

The later installed requalification reported a final median of 301.9 milliseconds, with p10 291.9 and p90 352.7 milliseconds across ten observations. Its direct-launch median was 63.4 milliseconds. The approximate added difference was therefore 238.5 milliseconds. These results are attributed to the final report because the named final raw CSV is absent from the supplied source set. [G01, G08]

Relative to 665.6 milliseconds, the reported final median decreased by about 54.6 percent. That percentage describes process-appearance time in these VM runs. It does not establish a 54.6 percent increase in download throughput, a reduction in Internet RTT, or an improvement in Tor's anonymity. The application's own execution timestamp is another measure and has separately reported values.

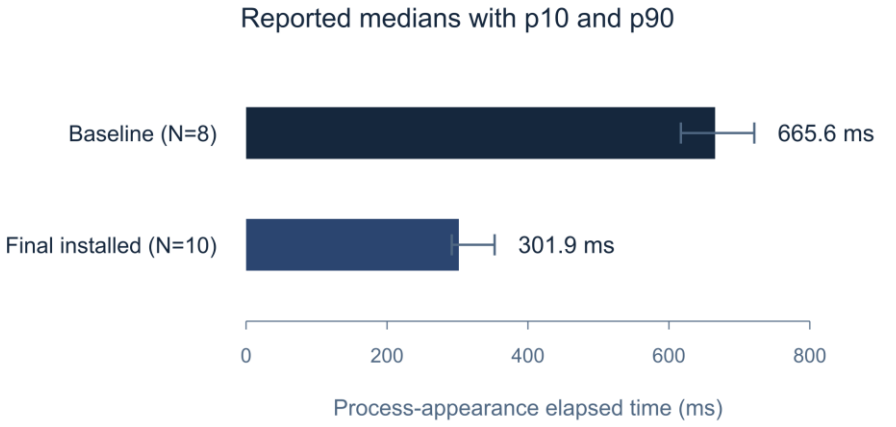


Figure 21.1. Reported launch process-appearance medians on the qualification VM. Baseline N=8; final installed N=10. Whiskers show reported p10 and p90 observations, not confidence intervals. The final raw CSV was unavailable to this manuscript. Sources: [G07], [G08], [G01].

The optimization record also reports reduced helper CPU per launch and essentially unchanged resident memory. Earlier staged measurements and final

installed measurements have different values, so they should not be merged into one row without identifying the run. The qualitative conclusion supported by those reports is reduced local launch work, rather than an unqualified reduction in every resource metric.

Rejected work is part of the investigation

The candidate list included folding a later registry-report round trip into the launch reply. A focused measurement found roughly 5.2 milliseconds for connect, greeting, and report, with the report operation itself around 2.7 milliseconds. The proposed protocol change would save a small fraction of an approximately 321-millisecond launch and require changing a versioned interface.

The project rejected the fold as unjustified by the measurable benefit and retained the fresh authoritative helper report. A variant reusing the connection would save even less. This decision illustrates a useful stopping rule: do not add interface risk and complexity merely because a profiler can name another operation. [G08]

Other candidates concerned the steady-state data path and DNS. A synthetic relay can move data far faster than the observed Tor workload while the network remains the bottleneck. Increasing its buffer may show a better synthetic number without improving user-visible transfer. Pooling DNS work might save local overhead while changing response correlation. The campaign explicitly deferred caching and did not weaken those properties to obtain a headline result. [G07]

Data-path optimization needs backpressure

Backpressure makes an upstream producer slow down when a downstream consumer cannot keep up. A relay with unlimited read-ahead can accumulate data in memory, improve one sender's apparent completion time, and worsen latency or stability elsewhere. Bounded buffers and correct readiness handling preserve control over that accumulation.

Batching socket operations can reduce syscalls per byte. Larger transfers can reduce bookkeeping, while excessively large batches can increase waiting for small interactive messages. Avoiding unnecessary copies can reduce memory bandwidth and CPU use, but shared buffers need correct ownership and lifetime. A stale reference can turn an optimization into memory corruption or disclosure.

Scheduling and wakeups also matter. Busy polling can reduce waiting while consuming CPU continuously. Sleeping too long saves CPU while adding latency. Parallelism can overlap independent setup but creates races if operations actually depend on one another. Each technique needs a workload and an invariant, rather than a generic instruction to maximize batching or concurrency.

Retest the property that could have changed

The post-optimization report records real-Tor APP, leakage, lifecycle, boot-guard, partial-construction, and full-gate checks. The create-path probes include partial ip-batch failure, nft application failure, early relay death, and a relay that never listens. The reported failures were refused without residue, with the silent relay bounded by a deadline. [G01]

Those tests address risks introduced by the changes: partial construction, premature readiness, and cleanup. They do not establish every future failure mode. The report also distinguishes a product session-exit defect from defects in the qualification harness itself. Fixing a harness can improve the reliability of evidence without changing the product's packet path.

The intended preserved properties include namespace confinement, the dead-end route, no masquerading, the original-destination relay contract, per-group identity information, DNS confinement, effective-policy comparison, and least privilege. The records report that the qualified candidate retained those properties under the campaign. The host trust boundary and the open evidence gaps remain.

The optimization belongs to the wrapper

Tor still builds and uses its privacy paths. Its relay count, cryptographic design, and external network capacity did not become cheaper because Ghostnector launches fewer local administration processes. The change reduces overhead in software that establishes the protected environment around Tor.

That is a substantial result precisely because it has a clear boundary. A reader can identify the altered operations, the measured outcome, and the guarantees that were rechecked. The central question can now be answered as an engineering problem with constraints, rather than a request for the name of a fast privacy product.

22 What can be masked at an acceptable cost

Specify the information first

We began with application bytes in memory and followed their movement through sockets, protocol state, routes, devices, and intermediaries. Every stage needed information to do its job. Some of that information can be hidden from a particular observer; some must remain available to the component currently forwarding or interpreting the traffic.

Masking content means preventing an unauthorized observer from reading the application data. Masking an origin means changing or concealing the relationship between the sender's network identity and a destination. Reducing metadata exposure means limiting visible timing, size, direction, names, or relationships. Providing anonymity means preserving uncertainty about a participant within the relevant observer's possible explanations. These are different objectives with different costs.

For a passive observer on an untrusted local network, authenticated application encryption can be sufficient for content confidentiality under trusted endpoints. It leaves destination and timing information. A VPN can hide the final network destinations from that local observer while allowing the VPN operator to associate the client with onward traffic. A multi-relay anonymity system distributes ordinary knowledge further, while retaining limits against correlation and application identification.

For an internal I2P destination or a Tor onion service, the architecture can conceal the service's ordinary network location from its client. That is another objective, with a different path from visiting a public HTTPS site through an exit. A comparison must preserve the service relationship before judging the performance.

The fastest feasible design depends on the adversary

Suppose an organization needs to prevent readable traffic on a shared access network while allowing its gateway to know destinations. Under that hypothetical trust model, a managed authenticated tunnel may satisfy the network objective. Adding several anonymous relays would solve a broader problem while adding cost the organization did not require.

Now suppose the gateway operator must not be able to associate the client's access identity with the destination. The first design no longer meets the requirement. Its low latency cannot make it the optimal solution to the second problem, because it is outside the feasible set. The feasible set is the collection of designs that satisfy the stated constraints.

Finally, suppose the adversary can observe all relevant entry and exit traffic and perform capable timing analysis. A low-latency overlay cannot honestly promise complete protection against that observer merely by adding another familiar label. Stronger resistance can require delay, cover traffic, application changes, or a different communication pattern, each with resource and usability consequences. The available evidence may not establish a feasible interactive design for an absolute requirement.

This explains why the central question has no universal product answer. Performance is optimized inside an explicit protection model. Replacing the model with a weaker one can lower cost, but the result must be named as a different decision.

Separate fundamental cost from avoidable work

Additional physical distance contributes propagation delay. Forwarding through independent participants adds availability dependencies and opportunities to queue. Establishing authenticated cryptographic state requires computation and communication. Hiding observable patterns can require extra bytes or delay. These costs follow from particular mechanisms or deployment conditions, though their precise amount is not fixed for every implementation.

Repeatedly starting the same administration tool, performing redundant state discovery, or waiting a full polling tick after readiness has already arrived are different costs. They arise from implementation choices. Reducing them can preserve the same external behavior if ordering, validation, and failure handling remain intact.

Ghostnector's launch optimization supplies evidence for that second category. The reported median reduction occurred in establishing the local APP environment. The same-instance HTTP audit does not establish an equally clear

steady-state network saving, and there is no basis here for inventing a Tor-versus-I2P benchmark. The evidence supports a narrower result that is still useful: local setup work can be reduced substantially while retaining the campaign's qualified confinement behavior. [G01, G07, G08, G18]

Optimize several outcomes honestly

A design has a performance profile rather than one speed number. Its relevant quantities may include startup latency, request latency, throughput, tail delay, CPU time, memory, and bandwidth consumed by cover or participation traffic. Compatibility and recovery effort are also costs, even when they do not appear on a stopwatch.

One design dominates another only if it improves relevant outcomes without worsening any required outcome or violating a constraint. Often neither dominates: one has lower latency, another lower memory, and a third a different trust assumption. A Pareto frontier is the set of options for which improving one chosen outcome would require worsening another within the considered choices. It is a way to describe remaining tradeoffs, not proof that every possible architecture has been discovered.

An engineering team can choose a priority, such as minimizing application startup while preserving specified isolation and keeping memory within a budget. It should state the budget and the preserved properties before measurement. Otherwise, a faster result can quietly come from allowing a direct fallback, using a warm cache only on one side, or removing the verification that would have detected a failed path.

The cost of failing closed belongs in that profile. A blocked machine may preserve confinement while interrupting work. Recovery can require local access. If a product chooses automatic reopening instead, it must explain which protected intent it has withdrawn and what traffic can now leave.

A repeatable decision process

The investigation can be organized into a short sequence without turning it into a product recipe:

1. Describe the observers, trusted components, and information that should remain hidden.

2. Draw every relevant path, including DNS, other protocol families, supporting-service exceptions, and failure states.

3. Select mechanisms that satisfy those constraints and state the remaining exposure.

4. Measure equivalent workloads with clear event boundaries and preserved raw observations.

5. Locate costs the implementation controls and change the smallest justified mechanism.

6. Recheck the affected protection and lifecycle properties, then report the measured outcome with its limits.

The order matters. A benchmark cannot supply a missing threat model. A successful ordinary request cannot establish confinement. A restrictive firewall cannot establish anonymity against application identifiers. A passing test suite cannot establish behavior it never observed.

What the case study establishes

The supplied project supports a concrete engineering account: an early design had falsified claims; later work corrected ordering and destination-recovery mechanisms; installed qualification exposed service and lifecycle failures; performance measurement found a local startup bottleneck; an optimization removed repeated work; and later reports describe requalification of the resulting candidate.

It also supports qualifications. Some raw logs are missing. A subset of configured checks can support the displayed protected state under its definition. Privileged host compromise remains outside the confinement claim. Direct observation of the earliest boot window remains an open evidence item in the claims document. The original HTTP analysis needs category-correct pairing, for which this manuscript supplies a separate reconstruction. [G01, G02, G03, G18]

Those limits do not need to be concealed to tell a coherent story. They explain how engineering knowledge is made: a mechanism is proposed, an observation can contradict it, the implementation changes, and the evidence is narrowed or extended. A record that preserves an unsuccessful approach lets another engineer avoid inheriting its assumptions.

The answer we can defend

Network traffic can be masked with less unnecessary cost by selecting protection for a specific observer model, enforcing the complete intended path, and optimizing the implementation around that path while preserving its security and privacy properties. Application encryption, tunnels, relays, onion services, and destination-oriented overlays supply different mechanisms. Their useful combination depends on what must remain hidden and from whom.

There is no supported universally optimal solution in the materials examined here. Some costs are consequences of the desired separation and some are removable local work. Measurements determine which is which only when the workload, baseline, and observation boundaries are comparable.

A future change should leave enough evidence for another person to repeat that judgment. Keep the implementation identity, policy, workload, timing boundaries, failures, and raw observations together. The next engineer should be able to run the same investigation and discover whether the same protection still holds.

Appendix A One request through the complete machine

The assumptions of the example

This is an illustrative packet journey, not a captured Ghostnector experiment. A laptop has IPv4 address 192.168.10.23/24 on Ethernet and a gateway at 192.168.10.1. A server is represented by documentation address 198.51.100.40. The application requests an HTTPS resource from a name under example.com. The resolver has an appropriate answer available. We assume no proxy or VPN for the first journey, correct service authentication, and a conventional TCP-based HTTP transport.

The chosen addresses and protocol path make each decision explicit. A real browser may instead use IPv6, reuse an existing connection, select HTTP/3, or find the response in a cache. Those possibilities do not invalidate the example; they identify a different path that should be drawn separately.

From the request to the gateway

The application first needs the service address. It asks its resolver interface, which can consult local naming rules and caches before sending a query. If a network query is needed, that query has its own socket, destination, route, and packet processing. It is not automatically part of the later HTTPS connection merely because the application caused both operations.

After receiving the server address, the application creates a TCP socket and requests connection to destination port 443. The operating system selects a local address and suitable temporary port under its routing and binding rules. A route lookup finds that 198.51.100.40 is outside the connected 192.168.10.0/24 prefix and selects the default gateway.

If the gateway's link address is not cached, the host sends an ARP request for 192.168.10.1. It does not broadcast a request for the remote server's MAC address. The gateway supplies its local mapping. The original TCP SYN can wait while the neighbor information is obtained, subject to the implementation's queues and limits.

The TCP implementation creates connection state and prepares a SYN with its initial sequence information and relevant options. IP supplies the source and

destination addresses and other network-layer fields. Ethernet frames the resulting packet for the local gateway. The Ethernet destination is the gateway's MAC address; the IP destination remains 198.51.100.40.

The driver provides descriptors for the NIC to read suitable buffers. The NIC transmits the encoded frame over the local link. Depending on offloads, some final checksum or segmentation work may happen in hardware. A capture taken before that work can differ from the wire representation in ways that need explanation rather than an immediate diagnosis of corruption. [R01, R03, R05, R06, R13, R14, R46]

Beyond the local network

The gateway receives the frame, interprets the packet, and selects an onward route. In this illustrative deployment it performs source address and port translation for the internal client, maintaining the mapping for return traffic. It supplies new link framing for its own next hop. The laptop's Ethernet addresses are no longer the link addresses used across the next routed network.

Each subsequent router forwards toward the destination under its current information and decrements the applicable hop allowance. It may queue the packet behind other traffic. A loop, failure, or exhausted allowance can cause a discard, with control-message feedback depending on conditions. The routers need not understand the HTTPS request that has not yet been sent.

The destination host receives the SYN and matches it to a listening TCP service. Its SYN-ACK returns along an available route, which need not be the identical reverse physical path. The gateway associates the reply with its translation state and restores the internal destination. The laptop's TCP implementation processes the reply and sends the concluding acknowledgment.

The established socket now represents transport state between the endpoints as exposed through the translation. A firewall on the path may retain its own flow state. These are several related records, not one global connection object shared by every device.

The secure application exchange

The application or its TLS library conducts the handshake through that stream. It validates the intended server identity and derives keys under the negotiated protocol. A certificate error changes what the application can safely conclude even if the TCP connection works perfectly. A successful TCP handshake is evidence of transport reachability, not of cryptographic identity.

The application then encodes its HTTP request. TLS protects the relevant application bytes. TCP carries them as an ordered stream, which may divide them among segments independently of the application's write boundaries. IP and the links carry those units through the network.

At the server, the NIC and driver make received data available to the stack. TCP validates and orders bytes, acknowledging transport progress. The TLS endpoint checks authentication tags and recovers valid plaintext. The HTTP server interprets the request and prepares a response. It may perform storage, computation, or internal service calls before any response byte returns.

That interval can dominate time to first byte. A client-side profiler that labels all waiting "network delay" would misattribute server processing and application dependencies. A complete explanation needs instrumentation or a bounded inference, rather than a name chosen for the gap.

The return response follows the corresponding processing in the laptop. Its process may not run immediately when data becomes available. Once it reads and decrypts the response, the browser may still need to parse, execute, fetch dependent resources, and render the page. Body completion and visible page completion remain different events. [R02, R13, R14, R16, R21]

What changes with a protected path

If the application uses an explicit SOCKS proxy, its first TCP connection is to that proxy. A SOCKS request carries the intended destination, and the onward connection becomes part of the proxy's work. Supplying a hostname through that interface changes the resolution location relative to the direct journey. The inner TLS relationship can still reach the intended service.

If a VPN accepts an inner IP packet through a virtual interface, the machine prepares an outer protected exchange to the VPN endpoint. The access network routes that outer exchange. The VPN gateway recovers the inner packet and forwards it. The original server can still receive authenticated TLS while seeing the gateway's connecting address.

If the APP path follows the final documented Ghostnector arrangement, the local connection is intercepted inside the application namespace. The namespace relay obtains the original destination there and sends it explicitly through SOCKS toward Tor. The default route's dead-end device prevents an ordinary external fallback when interception is absent. The application does not gain a direct host route simply because the host itself has Internet access. [G04, G11]

An I2P application takes a different branch. Its destination identity and naming interface lead to the local I2P router, which selects an outbound tunnel and

obtains the recipient's inbound reachability. Replacing 198.51.100.40 with a supposed I2P public IP would erase the architectural difference rather than explain it. [R39, R40, R41]

A practical reading of a failed request

If the first connection fails, ask which phase produced evidence. A DNS answer establishes that one resolution operation produced a result. A route establishes that the kernel has a forwarding choice. An ARP reply establishes a claimed link mapping. A local proxy accept establishes a local listener. None of those alone establishes a complete authenticated application exchange.

If a small response works and a large one stalls, examine packet-size and loss behavior as well as the application. If a direct request works but a protected one fails, distinguish unavailable privacy peers from local enforcement errors. If a supposedly prohibited request fails, establish that the destination and observer would have detected an allowed attempt.

The point of tracing one request is to make those distinctions available. A user-visible failure is one symptom produced by a sequence of dependent mechanisms. The evidence becomes useful when it identifies which dependency held, which failed, and which was never observed.

Appendix B Reading and extending the evidence

The source hierarchy

The manuscript uses the supplied repository at commit 17a03c4 as its project source boundary. The release-candidate report's post-optimization section names 9b0fe5d as the tested product baseline. The earlier f82adf9 campaign remains evidence for its own implementation and for unaffected behavior where the later report explicitly says so. Release metadata and a reported test baseline answer different questions.

The implementation plan and handoff records preserve earlier intentions and snapshots. They are useful for explaining why a design changed, but later decision amendments and applicable source take precedence for a current mechanism. The final claims document is especially important when its narrower language qualifies a broader summary in a release report.

Source hashes are retained in sources/source-manifest.json. They identify the local files available to this manuscript, not every file that existed on the qualification VM. The copied author brief remains in the same sources directory. Per-chapter source identifiers make the supporting material discoverable without requiring the book to become a collection of raw logs.

Four consequential source qualifications

First, the D-50 amendment changes the APP destination-recovery mechanism. Older text describing a direct redirect to the host-side transparent listener is historical. The final relay source and later report establish the namespace-local relay and explicit SOCKS path. [G01, G04, G11]

Second, the post-optimization report supersedes earlier launch medians as the final reported installed measurement. The 320.9-millisecond staged median and 301.9-millisecond final installed median belong to separate runs. The accessible staged log supports the former. The final named CSV is absent here, and the book attributes that latter value to the reports. The early log ending with zero usable product samples cannot independently support a baseline merely because the campaign cites a similarly named record. [G08, G10]

Third, the boot guard's tested ordering relative to a service barrier does not close every question about traffic before enforcement. The claims file keeps G6 open while recording the installed guard correction and focused qualification under G14. The book preserves both observations. Likewise, privileged attackers remain outside the host-trust boundary, and DHCPv6-only lease maintenance remains unsupported by the documented IPv4 DHCP exception. [G02]

Fourth, the HTTP scripts collide latency and throughput iteration identifiers. The manuscript's audit reconstructs sample rows from the retained log and filters the categories before pairing. It leaves the original scripts and reports unchanged. The derived CSV and benchmark-audit.json are new analysis artifacts, not original qualification records. [G09, G18]

A reproducible experiment record

A useful follow-up experiment begins by writing the claim in falsifiable terms. "No ordinary application UDP reaches this observer while this scope claims protection" identifies a traffic class, boundary, and time condition. "The tool is secure" does not identify a testable observation.

Record the code and installed binary identities, operating system, kernel, relevant services, configuration, network arrangement, and workload. Describe the baseline in enough detail to establish that it preserves the required privacy properties. A direct baseline and an equivalent-overlay baseline can both be useful, but they answer different questions and should have different labels.

Choose clock and event boundaries before collecting results. For a local elapsed interval, use a suitable monotonic clock. For cross-machine ordering, record the offset or synchronization evidence and its uncertainty. A packet close to an uncertain boundary should be classified accordingly, rather than rounded into the preferred state.

Run controls showing that listeners and observers work. Observe at the far side when the claim concerns actual boundary crossing. Keep the state history and packet history together. A sample counter paired with a later state can mistakenly assign pre-protection traffic to a protected period, as Ghostnector's historical D-25 oracle defect demonstrated. [G06]

Predefine sample categories and unique keys. A safe key includes the workload category as well as the iteration, or uses globally unique sample identifiers. Record failed attempts, exclusions, warmups, and cleanup. A throughput row should never become a latency partner because both happen to be numbered one.

Interpreting the result

Publish the distribution and the operational consequence. For a launch change, show the event being timed, the number of observations, typical elapsed time, observed spread, CPU accounting, and any failures. For a privacy claim, show what was attempted, what the boundary observed, what was expected to arrive, and which cases remained inconclusive.

An interval crossing zero can mean the experiment does not clearly resolve the effect under its assumptions. It does not establish identical implementations. An apparently large improvement can be real for one metric while irrelevant to another. A passing fake-service test can establish deterministic state behavior without establishing real public-network integration.

Keep interpretation separate from observation. "The relay route had four long outliers" describes data. "Circuit-isolation behavior is consistent with those outliers" proposes an explanation. A stronger causal claim needs a design that distinguishes that explanation from competing ones while preserving the relevant protection.

The project records are valuable partly because they retain these distinctions, including corrections when they were initially missed. Extending them should preserve the same discipline: a missing observation stays missing until a suitable experiment supplies it.

References

Using these references

R identifies external technical sources. G identifies project material. Bracketed identifiers in the chapters refer to these entries. For specifications, consult the relevant section and listed updates or errata when implementing a detail. A historical specification can explain the origin of a mechanism without establishing every current implementation choice. External sources were consulted on 24 October 2026.

Hardware operating systems and basic protocols

[R01] Linux kernel documentation. Dynamic DMA mapping Guide. Device-visible addresses, CPU addresses, and DMA ownership. Read source.

[R02] Linux kernel documentation. NAPI. Driver notification and budgeted polling. Read source.

[R03] Linux kernel documentation. struct sk_buff. Packet metadata, associated buffers, and cloning. Read source.

[R04] Linux kernel documentation. Scaling in the Linux Networking Stack. Hardware and software steering mechanisms. Read source.

[R05] D. Plummer. RFC 826, An Ethernet Address Resolution Protocol, 1982. Read source.

[R06] J. Postel, editor. RFC 791, Internet Protocol, 1981. The base IPv4 design, read with subsequent updates. Read source.

[R07] S. Deering and R. Hinden. RFC 8200, Internet Protocol Version 6 Specification, 2017. Read source.

[R08] T. Narten and others. RFC 4861, Neighbor Discovery for IP version 6, 2007. Read source.

[R09] R. Droms. RFC 2131, Dynamic Host Configuration Protocol, 1997. IPv4 DHCP behavior. Read source.

[R10] P. Mockapetris. RFC 1034, Domain Names Concepts and Facilities, 1987. Read source.

[R11] P. Mockapetris. RFC 1035, Domain Names Implementation and Specification, 1987. Both foundational DNS RFCs have extensive subsequent updates. Read source.

[R12] J. Postel. RFC 768, User Datagram Protocol, 1980. Read source.

[R13] Linux man-pages project. socket(7), the Linux socket interface. Read source.

[R14] W. Eddy, editor. RFC 9293, Transmission Control Protocol, 2022. Consolidated TCP base specification replacing RFC 793. Read source.

[R15] M. Allman, V. Paxson, and E. Blanton. RFC 5681, TCP Congestion Control, 2009. Classic algorithms and behavior, rather than a claim that every current sender uses an identical algorithm. Read source.

Application transport and security

[R16] R. Fielding, M. Nottingham, and J. Reschke, editors. RFC 9110, HTTP Semantics, and RFC 9112, HTTP/1.1, 2022. HTTP semantics. HTTP/1.1 framing.

[R17] M. Thomson and C. Benfield, editors. RFC 9113, HTTP/2, 2022. Read source.

[R18] J. Iyengar and M. Thomson, editors. RFC 9000, QUIC A UDP-Based Multiplexed and Secure Transport. M. Thomson and S. Turner. RFC 9001, Using TLS to Secure QUIC. J. Iyengar and I. Swett, editors. RFC 9002, QUIC Loss Detection and Congestion Control. All 2021. Transport. TLS integration. Recovery and congestion.

[R19] M. Bishop, editor. RFC 9114, HTTP/3, 2022. Read source.

[R20] H. Schulzrinne and others. RFC 3550, RTP A Transport Protocol for Real-Time Applications, 2003. M. Baugher and others. RFC 3711, The Secure Real-time Transport Protocol, 2004. RTP and RTCP. SRTP.

[R21] E. Rescorla. RFC 9846, The Transport Layer Security Protocol Version 1.3. The RFC Editor page identifies it as obsoleting RFC 8446. RFC 8446, published in 2018, remains the earlier TLS 1.3 reference. Current consulted specification. Earlier specification and status.

[R22] D. McGrew. RFC 5116, An Interface and Algorithms for Authenticated Encryption, 2008. Read source.

[R23] R. Arends and others. RFC 4033, DNS Security Introduction and Requirements, 2005. Read with the protocol's companion and updating specifications. Read source.

[R24] Z. Hu and others. RFC 7858, Specification for DNS over Transport Layer Security, 2016. Read source.

[R25] P. Hoffman and P. McManus. RFC 8484, DNS Queries over HTTPS, 2018. Read source.

[R26] Y. Rekhter, T. Li, and S. Hares, editors. RFC 4271, A Border Gateway Protocol 4, 2006. Read source.

[R27] P. Mohapatra and others. RFC 6811, BGP Prefix Origin Validation, 2013. Read source.

[R28] Netfilter project nftables wiki. Netfilter hooks. Packet paths and chain priorities. Read source.

[R29] Linux man-pages project. network_namespaces(7). Resources isolated by a network namespace. Read source.

[R30] Linux man-pages project. capabilities(7). Linux privilege sets and capability semantics. Read source.

[R31] M. Leech and others. RFC 1928, SOCKS Protocol Version 5, 1996. M. Leech. RFC 1929, Username/Password Authentication for SOCKS V5, 1996. Forwarding protocol. Authentication method.

[R32] S. Kent and K. Seo. RFC 4301, Security Architecture for the Internet Protocol, 2005. Read source.

Tor and I2P

[R33] Tor Project. A short introduction to Tor, Tor Specifications. Read source.

[R34] Tor Project. Creating circuits, Tor Specifications. Circuit handshakes and extension. Read source.

[R35] Tor Project. Path selection and constraints, and Circuit Creation Entry Guard Selection. Exact constraints and exceptions are version-dependent. Path selection. Entry guards.

[R36] Tor Project. How do Onion Services work; The introduction protocol; The rendezvous protocol. The overview is simplified; the protocol specifications govern exact behavior. Overview. Introduction. Rendezvous.

[R37] R. Dingledine, N. Mathewson, and P. Syverson. Tor The Second-Generation Onion Router, USENIX Security, 2004. Historical design and threat model. Original paper.

[R38] Tor Project. History. Read source.

[R39] I2P Project. I2P A Scalable Framework for Anonymous Communication. The consulted page identifies a 2003 origin and a 2025 update, with explicit historical limitations. Read source.

[R40] I2P Project. Tunnel Routing. The consulted page is marked updated March 2026 and accurate for 0.9.68. Read source.

[R41] I2P Project. The Network Database. RouterInfo, LeaseSets, floodfill behavior, and documented threats. The consulted page is marked updated March 2025 and accurate for 0.9.65. Read source.

[R42] I2P Project. Garlic Routing. Used for the message-bundling concept; historical algorithm names on this overview are qualified in the manuscript. Read source.

[R43] I2P Project. ECIES-X25519-AEAD-Ratchet. Modern end-to-end message-protection specification, distinguished from tunnel and transport layers. Read source.

[R44] I2P Project. Streaming Protocol. The consulted page is marked updated January 2026 and accurate for 0.9.68. Read source.

Additional mechanism references

[R45] F. Baker and G. Fairhurst, editors. RFC 7567, IETF Recommendations Regarding Active Queue Management, 2015. Read source.

[R46] Linux kernel documentation. Segmentation Offloads. TSO, GSO, GRO, and related mechanisms. Read source.

[R47] E. Rescorla, K. Oku, N. Sullivan, and C. A. Wood. RFC 9849, TLS Encrypted Client Hello. Read source.

[R48] D. Eastlake. RFC 6066, Transport Layer Security Extensions Extension Definitions, 2011. Server Name Indication, interpreted with later TLS and ECH specifications. Read source.

[R49] Tor Project. Using Tor with other browsers; Fingerprinting protections; Checking for DNS leaks with SOCKS and Tor. Browser and application boundaries. Other browsers. Fingerprinting. DNS behavior.

[R50] A. Retana and others. RFC 3021, Using 31-Bit Prefixes on IPv4 Point-to-Point Links, 2000. Read source.

[R51] R. Barnes and others. RFC 7568, Deprecating Secure Sockets Layer Version 3.0, 2015. Read source.

Ghostnector primary records

The paths below are relative to the supplied Ghostnector repository. The retained source manifest records hashes of available files. Logs named inside reports but absent here are not represented as directly inspected evidence.

[G01] docs/RELEASE-CANDIDATE-REPORT.md. Consolidated installed qualification and section 0 post-optimization requalification. Primary report for final campaign outcomes and tested product commit.

[G02] docs/PROTECTION-CLAIMS.md. Exact state semantics, confinement and availability claims, historical falsifications, scope, and remaining gaps. Its qualifications limit broader release-summary language.

[G03] docs/QUALIFICATION-NOTES.md. Native environment, installed integration failures, D-50 destination-recovery correction, profile transitions, and final requalification notes.

[G04] docs/M8-DECISIONS.md. APP topology, launcher boundaries, namespace experiment, and later D-50 amendment. Original host-side redirect language is historical where superseded.

[G05] docs/M9-DECISIONS.md. I2P policy, explicit refusals, canary requirement, fake-router versus real-router testing, and reported native qualification.

[G06] docs/M10-DECISIONS.md. GUI boundary and qualification prerequisites, including the D-25 timestamp-oracle correction.

[G07] perf/CAMPAIGN-RESULTS.md. Paired-baseline design, phase investigation, resource-method corrections, and optimization candidates. Its original paired HTTP statistics are qualified by the manuscript audit.

[G08] perf/APP-LAUNCH-OPTIMIZATION.md. Staged build measurements, reported final installed measurements, rejected IPC change, and preserved invariants.

[G09] perf/paired-http-20260928T143837Z.log and its summary.txt companion. Retained same-instance raw sample lines and original summary. scripts/lib/perf-paired-analysis.py and perf-paired-summary.py supply the analysis behavior examined in chapter 20.

[G10] perf/app-launch-profile-20260928T152100Z.log, app-launch-profile-20260928T161456Z.log, and app-launch-profile-20260928T162747Z.log. The first ends with no usable product samples; the second supports the staged 320.9 ms median; the third records the item-1 423.0 ms median. These do not substitute for the absent final installed CSV.

[G11] crates/ghostnector-appd/src/bin/relay.rs and crates/ghostnector-appd/src/backend.rs. Direct source for destination recovery, SOCKS forwarding, privilege drop, readiness, construction, and relay shutdown.

[G12] docs/RECOVERY.md. Local stand-down, boot override, rescue recovery, and the deliberate consequences of withdrawing protected intent.

[G13] docs/ADVERSARIAL-TEST-PLAN.md. Falsifiable cases, observation points, historical defect ledger, and qualification distinctions.

[G14] docs/HANDOFF-RC2.md. Historical M1-M7 architecture and evidence snapshot. Later records govern superseding behavior.

[G15] docs/IMPLEMENTATION-PLAN.md. Milestone intentions and original gates, distinguished from proof that every planned item was delivered.

[G16] docs/HANDOFF-M10.md. Inherited state model, scope distinctions, verification philosophy, and privileged component boundaries before later campaign amendments.

[G18] Manuscript source audit. sources/reconstructed-same-instance.csv and sources/benchmark-audit.json, generated by production/audit_sources.py from [G09]. Newly computed descriptive statistics for the book, not a new network experiment. The original project scripts and data were left unchanged.